# PUFFER

CONTENTS


| | | PAGE |
|---|---|---|
| 01 | package.json | 2 |
| 02 | data/local-installer-release.json | 4 |
| 03 | app/page.tsx | 5 |
| 04 | components/lab-workspace.tsx | 26 |
| 05 | components/puf-ranking.tsx | 36 |
| 06 | components/offtarget-workspace.tsx | 40 |
| 07 | components/local-downloads.tsx | 50 |
| 08 | lib/project.ts | 52 |
| 09 | lib/workflow-project.ts | 56 |
| 10 | lib/puf-ranking.ts | 60 |
| 11 | lib/offtarget-workflow.ts | 65 |
| 12 | lib/scientist-csv.ts | 70 |
| 13 | lib/workflow-service.ts | 76 |
| 14 | lib/local-downloads.ts | 82 |
| 15 | scripts/run-bounded.mjs | 87 |
| 16 | tests/scientist-csv.test.mjs | 88 |
| 17 | tests/offtarget-workflow.test.mjs | 98 |
| 18 | tests/workflow-service.test.mjs | 104 |
| 19 | LICENSE | 113 |






package.json

```
{
  "name": "puf-studio-igem",
  "version": "0.1.0",
  "private": true,
  "engines": {
    "node": ">=22.13.0"
  },
  "scripts": {
    "install:ci": "bash scripts/install-ci.sh",
    "dev": "WRANGLER_LOG_PATH=.wrangler/wrangler.log vite",
    "build": "bash scripts/build-verified.sh",
    "start": "WRANGLER_LOG_PATH=.wrangler/wrangler.log vinext start",
    "test": "npm run build && node --test tests/*.test.mjs",
    "lint": "bash scripts/sites-env.sh -- eslint . --ignore-pattern dist --ignore-pattern .next",
    "db:generate": "bash scripts/sites-env.sh -- drizzle-kit generate"
  },
  "dependencies": {
    "@base-ui/react": "^1.7.0",
    "@hookform/resolvers": "^5.7.1",
    "@shadcn/react": "^0.3.0",
    "class-variance-authority": "0.7.1",
    "clsx": "2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.4.0",
    "drizzle-orm": "0.45.2",
    "embla-carousel-react": "^8.6.0",
    "input-otp": "^1.4.2",
    "lucide-react": "^1.31.0",
    "next": "16.2.6",
    "next-themes": "^0.4.6",
    "radix-ui": "^1.6.7",
    "react": "19.2.6",
    "react-day-picker": "^10.0.1",
    "react-dom": "19.2.6",
    "react-hook-form": "^7.85.0",
    "react-resizable-panels": "^4.12.2",
    "recharts": "^3.8.0",
    "sonner": "^2.0.8",
    "tailwind-merge": "3.6.0",
    "vaul": "^1.1.2",
    "zod": "^3.25.76",
    "fflate": "0.7.4"
  },
  "devDependencies": {
    "@cloudflare/vite-plugin": "1.37.1",
    "@tailwindcss/postcss": "4.2.1",
    "@types/node": "22.19.19",
    "@types/react": "19.2.14",
    "@types/react-dom": "19.2.3",
    "@vitejs/plugin-react": "6.0.2",
    "@vitejs/plugin-rsc": "0.5.26",
    "drizzle-kit": "0.31.10",
    "eslint": "9.39.4",
    "eslint-config-next": "16.2.6",
    "react-server-dom-webpack": "19.2.6",
    "tailwindcss": "4.2.1",
```





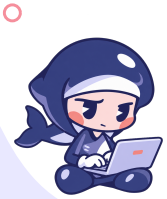
```
    "tw-animate-css": "^1.4.0",
    "typescript": "5.9.3",
    "vinext": "0.0.50",
    "vite": "8.0.13",
    "wrangler": "4.92.0"
  },
  "type": "module",
  "displayName": "PUF Studio \u00b7 RNA Editing Design"
}
```





data/local-installer-release.json

```
{
  "version": "0.2.0-preview",
  "fileName": "ORCA-oneclick-v0.2.0-preview-20261013.zip",
  "bytes": 204412,
  "sha256": "0f03a090bedfc4d93fa0b414efe5202d8aed4d8c3e892eabc5958c2887d82d25",
  "urls": {
    "windows": "/downloads/v0.2.0-preview/windows.zip",
    "macos": "/downloads/v0.2.0-preview/macos.zip"
  }
}
```





app/page.tsx

```tsx
'use client';
import { useLayoutEffect, useMemo, useRef, useState } from 'react';
import {
  ArrowUpRight,
  ArrowRight,
  Check,
  Dna,
  Download,
  FileText,
  Layers,
  Loader2,
  Play,
  Target,
  X,
} from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Textarea } from '@/components/ui/textarea';
import { Tabs, TabsList, TabsTrigger, TabsContent } from '@/components/ui/tabs';
import {
  Dialog,
  DialogContent,
  DialogTitle,
  DialogDescription,
} from '@/components/ui/dialog';
import { NativeSelect } from '@/components/ui/native-select';
import {
  Table,
  TableHeader,
  TableBody,
  TableRow,
  TableHead,
  TableCell,
} from '@/components/ui/table';
import {
  analyze,
  normalizeRNA,
  demoReferences,
  DEMO_SEQUENCE,
  DEFAULTS,
  modulePlan,
  validateLibrary,
  allCsites,
  recognitionWindows,
  editingSitesAfter,
  type Transcript,
  type ModuleLibrary,
  type PredictionSet,
  type Hit,
} from '@/lib/design';
import LabWorkspace from '@/components/lab-workspace';
import ArchitectureBuilder from '@/components/architecture-builder';
import { sourceOrder, withArchitecture, defaultOrder } from '@/lib/architecture';
import { assessmentReport, archivedAssessment } from '@/lib/assessment-report';
import type { StructureResult } from '@/lib/af3-analysis';
import { emptyLab, restoreProject, type LabState } from '@/lib/project';
```





```
import wtEvidence from '@/data/wt-structure-evidence.json';
import { buildDesignReport } from '@/lib/report';
import bundled from '@/data/puf-library.json';
import MethodsEvidence from '@/components/methods-evidence';
import PageNavigation from '@/components/page-navigation';
import IntroExperience, {
  AnimatedHeading,
  ScrollReveal,
} from '@/components/intro-experience';
import PufRanking from '@/components/puf-ranking';
import LocalDownloads from '@/components/local-downloads';
import {
  validateRankResult,
  type RankCandidate,
  type RankResult,
} from '@/lib/puf-ranking';
import {
  emptyOffTargetState,
  resultMatchesInputs,
  predictionsFromResult,
  type OffTargetState,
} from '@/lib/offtarget-workflow';
import { buildScientistCsv } from '@/lib/scientist-csv';
import './work-flow.css';
import { serializeWorkflow, restoreWorkflow } from '@/lib/workflow-project';
const builtIn = validateLibrary(bundled as ModuleLibrary),
  demoRefs = demoReferences();
const initial = analyze(DEMO_SEQUENCE, 76, 'TARGET-DEMO', demoRefs, DEFAULTS);
function download(name: string, data: string, type = 'text/plain') {
  const u = URL.createObjectURL(new Blob([data], { type }));
  const a = document.createElement('a');
  a.href = u;
  a.download = name;
  document.body.appendChild(a);
  a.click();
  a.remove();
  setTimeout(() => URL.revokeObjectURL(u), 1000);
}
function Bases({ sequence, reference }: { sequence: string; reference?: string }) {
  return (
    <span className="bases">
      {sequence.split('').map((b, i) => (
        <span
          key={i}
          className={`base base-${b} ${reference && b !== reference[i] ? 'mismatch' : ''}`}
        >
          {b}
        </span>
      ))}
    </span>
  );
}
function Pill({
  children,
  tone = 'green',
}: {
  children: React.ReactNode;
  tone?: string;
}) {
  return <span className={`pill ${tone}`}>{children}</span>;
}
```





```
export default function Home() {
  const [sequence, setSequence] = useState(DEMO_SEQUENCE),
    [position, setPosition] = useState(76),
    [targetId, setTargetId] = useState('TARGET-DEMO');
  const [length, setLength] = useState(12),
    [windowStart, setWindowStart] = useState<number | null>(63);
  const [refs, setRefs] = useState<Transcript[]>(demoRefs),
    [refName, setRefName] = useState('Synthetic example · 5 transcripts');
  const [result, setResult] = useState({
    candidates: initial,
    sequence: DEMO_SEQUENCE,
    position: 76,
    targetId: 'TARGET-DEMO',
    referenceName: 'Synthetic example · 5 transcripts',
    references: demoRefs,
    settings: DEFAULTS,
  });
  const [selected, setSelected] = useState('R12-D2'),
    [tab, setTab] = useState('design'),
    [mode, setMode] = useState('overview'),
    [busy, setBusy] = useState(false),
    [error, setError] = useState(''),
    [message, setMessage] = useState(''),
    [dataOpen, setDataOpen] = useState(false);
  // Reset after the Methods view mounts, before it is painted at the previous scroll position.
  useLayoutEffect(() => {
    if (mode === 'methods') window.scrollTo({ top: 0, left: 0, behavior: 'instant' });
  }, [mode]);
  const [library, setLibrary] = useState<ModuleLibrary>(builtIn),
    [predictions, setPredictions] = useState<PredictionSet | null>(null),
    [overrides, setOverrides] = useState<Record<number, string>>({});
  const [experience, setExperience] = useState('beginner');
  const [structureEvidence, setStructureEvidence] =
    useState<StructureResult[]>(wtEvidence);
  const [lab, setLab] = useState<LabState>(structuredClone(emptyLab));
  const [rankCandidates, setRankCandidates] = useState<RankCandidate[]>([]),
    [rankResult, setRankResult] = useState<RankResult | null>(null);
  const [offtargetState, setOfftargetState] = useState<OffTargetState>({
    ...emptyOffTargetState,
  });
  const [projectRevision, setProjectRevision] = useState(0);
  const projectInput = useRef<HTMLInputElement>(null);
  const resultsRef = useRef<HTMLElement>(null);
  function goToStep(next: string) {
    setTab(next);
    requestAnimationFrame(() => {
      resultsRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' });
      resultsRef.current
        ?.querySelector<HTMLButtonElement>('[data-slot=tabs-trigger][data-state=active]')
        ?.focus({ preventScroll: true });
    });
  }
  const windows = useMemo(() => {
    try {
      return recognitionWindows(normalizeRNA(sequence), position, length);
    } catch {
      return [];
    }
  }, [sequence, position, length]);
  const draftWindow = windows.find((w) => w.start === windowStart) || windows[0];
  const c = result.candidates.find((x) => x.id === selected) || result.candidates[0];
```





```
  const plan = useMemo(() => modulePlan(c, library, overrides), [c, library, overrides]);
  const workflowTarget = {
    id: result.targetId,
    sequence: result.sequence,
    position: result.position,
    motif: c.motif,
    bindingStart: c.start,
  };
  const activeOfftarget = resultMatchesInputs(offtargetState, workflowTarget)
    ? offtargetState.result
    : null;
  const effectivePredictions = useMemo(
    () => (activeOfftarget ? predictionsFromResult(activeOfftarget) : predictions),
    [activeOfftarget, predictions],
  );
  const scoreMap = useMemo(
    () => new Map(predictions?.predictions.map((p) => [p.context, p.score]) || []),
    [predictions],
  );
  let activeRank: RankResult | null = null;
  try {
    if (rankResult)
      activeRank = validateRankResult(rankResult, rankCandidates, rankResult.source);
  } catch {
    /* Input-bound results only. */
  }
  const dirty =
    sequence !== result.sequence ||
    position !== result.position ||
    targetId !== result.targetId ||
    refs !== result.references ||
    length !== result.settings.recognitionLength;
  const sites = useMemo(
    () =>
      allCsites(
        result.references.some((r) => r.id === result.targetId)
          ? result.references
          : [{ id: result.targetId, sequence: result.sequence }, ...result.references],
      ),
    [result],
  );
  const bystanders = editingSitesAfter(result.sequence, c.end).filter(
    (s) => s.position !== result.position,
  );
  const viewStart = Math.max(0, c.start - 5),
    viewEnd = Math.min(result.sequence.length, c.end + 9);
  function saveProject() {
    if (dirty) {
      setMode('workbench');
      goToStep('design');
      setMessage('Apply changes before saving.');
      return;
    }
    download(
      'puf-studio-project.json',
      JSON.stringify(
        {
          kind: 'puf-studio-project',
          version: 1,
          target: {
            id: result.targetId,
```





```
            sequence: result.sequence,
            position: result.position,
          },
          length: c.n,
          windowStart: c.start,
          referenceName: result.referenceName,
          references: result.references,
          library,
          predictions,
          overrides,
          lab,
          structureEvidence,
          experience,
          workflow: serializeWorkflow(
            {
              candidates: rankCandidates,
              ranking: activeRank,
              offtarget: offtargetState,
            },
            workflowTarget,
          ),
        },
        null,
        2,
      ),
      'application/json',
    );
  }
  async function openProject(file?: File) {
    if (!file) return;
    try {
      if (file.size > 40000000) throw Error('Project limit: 40 MB.');
      const raw = JSON.parse(await file.text());
      const { p, candidates, chosen, settings } = restoreProject(raw);
      const workflow = restoreWorkflow(raw.workflow, {
        id: p.target.id,
        sequence: p.target.sequence,
        position: p.target.position,
        motif: chosen.motif,
        bindingStart: chosen.start,
      });
      setProjectRevision((n) => n + 1);
      setRankCandidates(workflow.candidates);
      setRankResult(workflow.ranking);
      setOfftargetState(workflow.offtarget);
      setSequence(p.target.sequence);
      setPosition(p.target.position);
      setTargetId(p.target.id);
      setLength(p.length);
      setWindowStart(chosen.start);
      setRefs(p.references);
      setRefName(p.referenceName);
      setResult({
        candidates,
        sequence: p.target.sequence,
        position: p.target.position,
        targetId: p.target.id,
        referenceName: p.referenceName,
        references: p.references,
        settings,
      });
```





```
      setSelected(chosen.id);
      setLibrary(p.library);
      setPredictions(p.predictions);
      setOverrides(p.overrides);
      setLab(p.lab);
      setExperience(p.experience || 'beginner');
      setStructureEvidence(p.structureEvidence || []);
      setError('');
      setMessage('Project loaded.');
      setMode('workbench');
      goToStep('design');
    } catch (e) {
      setError((e as Error).message);
      setMode('workbench');
    }
  }
  function hitScore(h: Hit) {
    const scores = h.editingSites.flatMap((s) =>
      s.context && scoreMap.has(s.context) ? [scoreMap.get(s.context)!] : [],
    );
    return scores.length ? Math.max(...scores).toFixed(3) : '—';
  }
  async function run(next = 'design') {
    setError('');
    setMessage('');
    setBusy(true);
    await new Promise((r) => setTimeout(r, 30));
    try {
      const seq = normalizeRNA(sequence);
      const id = sequence.trim().startsWith('>')
        ? sequence.trim().split(/\s/)[0].slice(1)
        : targetId;
      const settings = { recognitionLength: length, mismatch: 2 };
      if (length >= 9 && length <= 11 && !library.sourceOrders?.[length])
        setLibrary(withArchitecture(library, length, defaultOrder(length)));
      const candidates = analyze(seq, position, id, refs, settings);
      const chosen =
        candidates.find((a) => a.start === draftWindow?.start) ||
        candidates.find((a) => a.offset === 2) ||
        candidates[0];
      setSequence(seq);
      setTargetId(id);
      setResult({
        candidates,
        sequence: seq,
        position,
        targetId: id,
        referenceName: refName,
        references: refs,
        settings,
      });
      setSelected(chosen.id);
      setWindowStart(chosen.start);
      setOverrides({});
      goToStep(next);
      setMode('workbench');
    } catch (e) {
      setError((e as Error).message);
    } finally {
      setBusy(false);
    }
```





```
  }
  function choose(id: string) {
    const candidate = result.candidates.find((a) => a.id === id);
    setSelected(id);
    if (candidate && !dirty) setWindowStart(candidate.start);
    setOverrides({});
  }
  async function loadLibrary(file: File | undefined) {
    if (!file) return;
    setError('');
    try {
      if (file.size > 10000000) throw new Error('File limit: 10 MB.');
      setLibrary(validateLibrary(JSON.parse(await file.text())));
      setOverrides({});
      setMessage('Library loaded.');
    } catch (e) {
      setError((e as Error).message);
    }
  }
  function reportData() {
    return {
      schemaVersion: '3.2',
      archivedAssessment: archivedAssessment(plan.sequence),
      repeatSourceOrder: sourceOrder(library, c.n),
      structureAssessments: assessmentReport(structureEvidence, plan.sequence),
      createdAt: new Date().toISOString(),
      analysisScope: result.referenceName,
      referenceSequences: result.references,
      target: {
        id: result.targetId,
        sequence: result.sequence,
        coordinate1: result.position,
      },
      settings: {
        recognitionLength: c.n,
        bindingInterval1: [c.start, c.end],
        editingOffsetFromBindingEnd: c.offset,
        editingOffsets: [2, 3, 4, 5],
        mismatch: 2,
      },
      selectedDesign: c,
      alternativeWindows: result.candidates,
      moduleLibraryVersion: library.version,
      trmTransferEvidence: library.transferPolicy || null,
      scaffoldEvidence: plan.scaffold?.evidence || null,
      scaffoldId: plan.scaffold?.id || null,
      backgroundSites: sites,
      modulePlan: plan.rows.map((r) => ({
        repeat: r.repeat,
        sourceRepeat: sourceOrder(library, c.n)[r.repeat - 1],
        base: r.base,
        module: r.selected,
      })),
      pufProtein: plan.sequence,
      lamar: predictions,
      pufRanking: activeRank,
      proteinCandidates: rankCandidates,
      offtargetAnalysis: activeOfftarget,
      measuredBindingFraction: null,
      predictedEditingFraction: null,
      localBystanders: bystanders,
```





```
      limitations: [
        'The +2–5 preference derives from CU5.1/CU5.17 ePUF10 results and is a design prior for ot
her architectures.',
        'No built-in 9–11R scaffold has been supplied.',
        'LAMAR ranks RNA contexts independently of PUF identity.',
        'Whole-transcriptome inference requires a server reference and model service.',
      ],
    };
  }
  function exportReport() {
    download(
      'PUF-Design-Report.html',
      buildDesignReport(reportData()),
      'text/html;charset=utf-8',
    );
  }
  function exportRawReport() {
    download(
      'puf-studio-report.json',
      JSON.stringify(reportData(), null, 2),
      'application/json',
    );
  }
  function exportHits() {
    download(
      'puf-studio-similar-sites.tsv',
      'transcript\tbinding_start_1based\tmotif\tmismatches\tdownstream_C_2to5\tmax_imported_lamar_
score\n' +
        c.hits
          .map((h) =>
            [
              h.transcript,
              h.start,
              h.sequence,
              h.mismatches,
              h.editingSites.map((s) => s.position).join(';') || 'NA',
              hitScore(h),
            ].join('\t'),
          )
          .join('\n'),
    );
  }
  function exportScientistCsv() {
    if (dirty) {
      setMessage('Apply changes before exporting.');
      goToStep('design');
      return;
    }
    const candidates = [...rankCandidates];
    if (
      plan.sequence &&
      !candidates.some((candidate) => candidate.protein === plan.sequence)
    ) {
      let id = 'selected-' + c.id,
        index = 1;
      while (candidates.some((candidate) => candidate.id === id))
        id = 'selected-' + c.id + '-' + index++;
      candidates.push({ id, protein: plan.sequence });
    }
    const ranks = candidates.map((candidate) => ({
      candidateId: candidate.id,
```





```
      protein: candidate.protein,
      rank: null,
      score: null,
      esm2Score: null,
      c295Hat: null,
      c388Hat: null,
      c871Hat: null,
      modelVersion: '',
      scope: 'fixed-assay-C295-C388-C871',
      source: 'website-design',
      status: 'not_scored',
    }));
    const csv = buildScientistCsv({
      target: {
        id: result.targetId,
        sequence: result.sequence,
        position: result.position,
      },
      selected: { ...c, hits: [] },
      plan,
      libraryVersion: library.version,
      reference: { name: '', species: '', kind: 'unspecified' },
      esmRanks: ranks,
      activeProteinCandidateId:
        candidates.find((candidate) => candidate.protein === plan.sequence)?.id ?? null,
      predictions: null,
    });
    download('ORCA-scientist-summary.csv', csv, 'text/csv;charset=utf-8');
  }
  function reset() {
    setSequence(DEMO_SEQUENCE);
    setPosition(76);
    setTargetId('TARGET-DEMO');
    setLength(12);
    setWindowStart(63);
    setRefs(demoRefs);
    setRefName('Synthetic example · 5 transcripts');
    setMessage('Example loaded. Apply to update.');
    setError('');
  }
  const stepTabs = [
    ['design', 'RNA', Target],
    ['optimize', 'PUF', Dna],
    ['report', 'Export', FileText],
  ] as const;
  const overviewSteps = [
    ['design', 'Target RNA', Target],
    ['optimize', 'Build PUF', Dna],
    ['report', 'Export', FileText],
  ] as const;
  function navigate(next: string) {
    if (busy) return;
    if (dirty && next !== 'design') {
      goToStep('design');
      setMessage('Apply changes in RNA.');
      return;
    }
    goToStep(next);
  }
  function continueToTrm() {
    if (dirty) void run('optimize');
```





```
    else goToStep('optimize');
  }
  function StepActions({
    next,
    label,
    previous,
  }: {
    next?: string;
    label: string;
    previous?: string;
  }) {
    return (
      <div className="step-actions">
        {previous && (
          <Button variant="outline" onClick={() => navigate(previous)}>
            Back · {previous === 'design' ? 'RNA' : 'PUF'}
          </Button>
        )}
        <Button
          className="next-step"
          disabled={busy}
          onClick={() =>
            next === 'optimize' && tab === 'design'
              ? continueToTrm()
              : next
                ? navigate(next)
                : exportReport()
          }
        >
          {label}
          {next ? <ArrowRight size={16} /> : <Download size={16} />}
        </Button>
      </div>
    );
  }
  return (
    <IntroExperience>
      <div className={`app-shell ${mode === 'overview' ? '' : 'minimal-ui'}`}>
        <header className="topbar">
          <a href="/" className="brand">
            <span className="brand-mark">
              <Dna size={24} />
            </span>
            <b>SUSTech</b>
          </a>
          <PageNavigation value={mode} onChange={setMode} />
        </header>
        <main className={`main ${mode === 'overview' ? 'overview-main' : ''}`}>
          {mode === 'overview' && (
            <div className="page-heading">
              <div>
                <AnimatedHeading />
              </div>
            </div>
          )}
          {mode === 'overview' ? (
            <section className="overview-page">
              <ScrollReveal className="start-reveal">
                <button
                  type="button"
                  className="start-orbit"
```





```
                  onClick={() => {
                    setMode('workbench');
                    goToStep('design');
                  }}
                >
                  <span>Start</span>
                  <ArrowUpRight size={24} aria-hidden="true" />
                </button>
              </ScrollReveal>
              <ScrollReveal className="journey-reveal">
                <div className="overview-section-heading">
                  <h2>A complete design journey.</h2>
                  <button className="text-button" onClick={() => setMode('methods')}>
                    Methods & Evidence <ArrowUpRight size={16} />
                  </button>
                </div>
                <div className="journey-grid">
                  {overviewSteps.map(([key, title, Icon], i) => (
                    <button
                      key={key}
                      onClick={() => {
                        setMode('workbench');
                        navigate(key);
                      }}
                    >
                      <span className="journey-number">0{i + 1}</span>
                      <Icon size={24} />
                      <h3>{title}</h3>
                      <p>
                        {
                          [
                            'Choose the RNA window for your target cytidine.',
                            'Build PUF repeats and save protein candidates.',
                            'Download your CSV and the local application.',
                          ][i]
                        }
                      </p>
                      <ArrowUpRight size={18} />
                    </button>
                  ))}
                </div>
              </ScrollReveal>
            </section>
          ) : mode === 'methods' ? (
            <MethodsEvidence
              engineering={
                <LabWorkspace
                  state={lab}
                  onChange={setLab}
                  protein={plan.sequence}
                  jobTarget={{
                    id: result.targetId,
                    sequence: result.sequence,
                    position: result.position,
                    motif: c.motif,
                  }}
                  onPredictions={setPredictions}
                />
              }
            />
          ) : (
```





```
        <>
          <div className="work-heading">
            <div>
              <h1>Work</h1>
            </div>
            <div className="work-heading-actions">
              <input
                ref={projectInput}
                type="file"
                accept=".json"
                hidden
                onChange={(e) => {
                  void openProject(e.target.files?.[0]);
                  e.target.value = '';
                }}
              />
              <Button variant="outline" onClick={() => projectInput.current?.click()}>
                Open
              </Button>
              <Button variant="outline" disabled={dirty} onClick={saveProject}>
                Save
              </Button>
              <Button variant="outline" onClick={() => setDataOpen(true)}>
                Data
              </Button>
              <Button disabled={dirty} onClick={exportScientistCsv}>
                <Download size={15} />
                CSV
              </Button>
            </div>
          </div>
          <div className="workspace">
            <aside className="input-panel panel">
              {tab === 'design' ? (
                <>
                  <div className="panel-heading">
                    <div className="section-title">
                      <Target size={18} />
                      <h2>Target</h2>
                    </div>
                    <Pill tone="neutral">C → U</Pill>
                  </div>
                  <div className="input-body simple-input">
                    <div className="field-head">
                      <label htmlFor="rna">
                        RNA <span>5′ → 3′</span>
                      </label>
                      <button className="text-button" onClick={reset}>
                        Example <ArrowUpRight size={12} />
                      </button>
                    </div>
                    <Textarea
                      id="rna"
                      value={sequence}
                      onChange={(e) => {
                        setSequence(e.target.value);
                        setWindowStart(null);
                      }}
                      spellCheck={false}
                      className="sequence-input"
                    />
```





```
                    <div className="input-caption">
                      <span>RNA, DNA or single-record FASTA</span>
                    </div>
                    <label htmlFor="target-transcript-id">Transcript ID</label>
                    <Input
                      id="target-transcript-id"
                      value={targetId}
                      maxLength={200}
                      onChange={(e) => setTargetId(e.target.value)}
                    />
                    <div className="form-row">
                      <div>
                        <label htmlFor="position">Target C</label>
                        <Input
                          id="position"
                          type="number"
                          min={1}
                          value={Number.isNaN(position) ? '' : position}
                          onChange={(e) => {
                            setPosition(Number(e.target.value));
                            setWindowStart(null);
                          }}
                        />
                      </div>
                      <div className="coordinate-hint">
                        <b>1-based</b>
                      </div>
                    </div>
                    <div className="form-divider" />
                    <label htmlFor="recognition-length">Length</label>
                    <NativeSelect
                      id="recognition-length"
                      value={length}
                      onChange={(e) => {
                        setLength(+e.target.value);
                        setWindowStart(null);
                      }}
                    >
                      {[8, 9, 10, 11, 12].map((n) => (
                        <option key={n} value={n}>
                          {n} nt
                        </option>
                      ))}
                    </NativeSelect>
                    <label htmlFor="binding-interval">Window</label>
                    <NativeSelect
                      id="binding-interval"
                      value={draftWindow?.start ?? ''}
                      onChange={(e) => {
                        const w = windows.find((w) => w.start === +e.target.value);
                        setWindowStart(+e.target.value);
                        if (w && !dirty) choose(w.id);
                      }}
                      disabled={!windows.length}
                    >
                      {windows.length ? (
                        windows.map((w) => (
                          <option key={w.id} value={w.start}>
                            {w.start}–{w.end} · {w.motif}
                          </option>
                        ))
```





```
                        ) : (
                          <option value="">Enter a valid target sequence</option>
                        )}
                      </NativeSelect>
                      {draftWindow && (
                        <div className="chosen-window">
                          <Bases sequence={draftWindow.motif} />
                          <small>Target C: +{draftWindow.offset} nt</small>
                        </div>
                      )}
                      <div className="editing-rule">
                        <span className="orange-square" />
                        <p>
                          <b>Editing window: +2–5 nt</b>
                        </p>
                      </div>
                      <Button
                        className="run-button"
                        onClick={() => run()}
                        disabled={busy}
                      >
                        {busy ? (
                          <Loader2 className="spin" size={17} />
                        ) : (
                          <Play size={16} />
                        )}{' '}
                        {busy ? 'Applying…' : 'Apply'}
                        <ArrowRight size={16} />
                      </Button>
                    </div>
                  </>
                ) : (
                  <>
                    <div className="panel-heading">
                      <div className="section-title">
                        <Target size={18} />
                        <h2>Target</h2>
                      </div>
                      <Pill>{c.n} nt</Pill>
                    </div>
                    <div className="fixed-target">
                      <span>
                        {result.targetId} · C{result.position}→U
                      </span>
                      <h3>
                        Interval {c.start}–{c.end}
                      </h3>
                      <Bases sequence={c.motif} />
                      <p>Target C: +{c.offset} nt</p>
                      <Button variant="outline" onClick={() => goToStep('design')}>
                        Edit RNA
                      </Button>
                    </div>
                  </>
                )}
              </aside>
              <section className="results-area" ref={resultsRef}>
                <Tabs value={tab} onValueChange={navigate}>
                  <TabsList className="workflow-tabs" aria-label="Design workflow">
                    {stepTabs.map(([key, title, Icon], i) => (
                      <TabsTrigger key={key} value={key}>
```





```
                        <span className="step-number">0{i + 1}</span>
                        <span>{title}</span>
                        <Icon className="step-icon" size={16} />
                      </TabsTrigger>
                    ))}
                  </TabsList>
                  {dirty && (
                    <div className="notice amber">
                      Inputs changed. Apply in step 01 to update your design.
                    </div>
                  )}
                  {error && (
                    <div className="notice error" role="alert">
                      {error}
                      <button aria-label="Dismiss error" onClick={() => setError('')}>
                        <X size={14} />
                      </button>
                    </div>
                  )}
                  {message && (
                    <div className="notice" role="status">
                      {message}
                      <button
                        aria-label="Dismiss message"
                        onClick={() => setMessage('')}
                      >
                        <X size={14} />
                      </button>
                    </div>
                  )}
                  <TabsContent value="design">
                    <section className="panel target-view">
                      <div className="panel-heading">
                        <div>
                          <h2>Window</h2>
                        </div>
                        <Pill>
                          {c.n} nt · {c.start}–{c.end}
                        </Pill>
                      </div>
                      <div className="sequence-viewport">
                        <div className="sequence-track">
                          <span className="sequence-direction">5′</span>
                          {Array.from(
                            { length: viewEnd - viewStart },
                            (_, i) => viewStart + i,
                          ).map((i) => (
                            <div
                              key={i}
                              className={`nucleotide ${
                                i === result.position - 1
                                    ? 'edit'
                                    : i >= c.start - 1 && i < c.end
                                      ? 'bound'
                                      : i + 1 >= c.end + 2 && i + 1 <= c.end + 5
                                        ? 'editing-zone'
                                        : ''
                              }`}
                            >
                              <small>
                                {(i + 1) % 5 === 0 || i === result.position - 1
```





```
                                ? i + 1
                                : '·'}
                            </small>
                            <b>{result.sequence[i]}</b>
                            <span>
                              {i === result.position - 1
                                ? 'C→U'
                                : i >= c.start - 1 && i < c.end
                                  ? `P${c.end - i}`
                                  : i + 1 > c.end && i + 1 <= c.end + 5
                                    ? `+${i + 1 - c.end}`
                                    : ''}
                            </span>
                          </div>
                        ))}
                        <span className="sequence-direction">3′</span>
                      </div>
                    </div>
                    <div className="sequence-legend">
                      <span>
                        <i className="teal-square" />
                        Recognition · {c.start}–{c.end}
                      </span>
                      <span>
                        <i className="orange-square" />
                        Editing preference · {c.end + 2}–{c.end + 5}
                      </span>
                    </div>
                  </section>
                  <section className="panel secondary-panel">
                    <div className="panel-heading">
                      <div className="section-title">
                        <Layers size={18} />
                        <h2>Windows</h2>
                        <span className="count">{result.candidates.length}</span>
                      </div>
                    </div>
                    <Table>
                      <TableHeader>
                        <TableRow>
                          <TableHead>Interval</TableHead>
                          <TableHead>RNA · 5′→3′</TableHead>
                          <TableHead>Choose</TableHead>
                        </TableRow>
                      </TableHeader>
                      <TableBody>
                        {[...result.candidates]
                          .sort((a, b) => a.offset - b.offset)
                          .map((a) => (
                            <TableRow
                              key={a.id}
                              className={a.id === c.id ? 'selected-row' : ''}
                            >
                              <TableCell>
                                <b>
                                  {a.start}–{a.end}
                                </b>
                                <small className="subtext">
                                  Target C: +{a.offset} nt
                                </small>
                              </TableCell>
```





```
                                <TableCell>
                                  <Bases sequence={a.motif} />
                                </TableCell>
                                <TableCell>
                                  <button
                                    className={`choose-button ${c.id === a.id ? 'chosen' : ''}`}
                                    aria-label={`Select interval ${a.start} to ${a.end}`}
                                    disabled={dirty || busy}
                                    onClick={() => choose(a.id)}
                                  >
                                    {c.id === a.id ? (
                                      <Check size={15} />
                                    ) : (
                                      <ArrowRight size={15} />
                                    )}
                                  </button>
                                </TableCell>
                              </TableRow>
                            ))}
                          </TableBody>
                        </Table>
                      </section>
                      <StepActions next="optimize" label="Next · PUF" />
                    </TabsContent>
                    <TabsContent value="optimize" forceMount hidden={tab !== 'optimize'}>
                      <ArchitectureBuilder
                        candidate={c}
                        library={library}
                        overrides={overrides}
                        level={experience}
                        onLevel={setExperience}
                        onLibrary={setLibrary}
                        onOverrides={setOverrides}
                        evidence={structureEvidence}
                        onEvidence={setStructureEvidence}
                        rna={result.sequence.slice(
                          Math.max(0, c.start - 1),
                          Math.min(result.sequence.length, c.end + 5),
                        )}
                      />
                      <PufRanking
                        key={projectRevision}
                        protein={plan.sequence}
                        disabled={dirty}
                        candidates={rankCandidates}
                        onCandidates={setRankCandidates}
                        result={activeRank}
                        onResult={setRankResult}
                      />
                      <StepActions
                        previous="design"
                        next="report"
                        label="Next · Export"
                      />
                    </TabsContent>
                    <TabsContent value="report">
                      <section className="panel report-panel">
                        <div className="panel-heading">
                          <div>
                            <h2>Export</h2>
                          </div>
```





```
                        <Button disabled={dirty} onClick={exportScientistCsv}>
                          <Download size={16} />
                          Download CSV
                        </Button>
                      </div>
                      <details className="report-details">
                        <summary>Design details</summary>
                        <div className="report-summary">
                          <div>
                            <span>Target</span>
                            <b>
                              {result.targetId} · C{result.position}→U
                            </b>
                          </div>
                          <div>
                            <span>Window</span>
                            <b>
                              {c.start}–{c.end} · {c.n} nt · target at +{c.offset}
                            </b>
                          </div>
                          <div>
                            <span>RNA</span>
                            <code>{c.motif}</code>
                          </div>
                          <div>
                            <span>Scaffold / modules</span>
                            <b>
                              {plan.scaffold?.id || 'Not supplied'} / {library.version}
                            </b>
                          </div>
                        </div>
                      </details>
                      <div className="protein-output">
                        <div className="field-head">
                          <h3>Protein</h3>
                          <Button
                            variant="outline"
                            disabled={!plan.sequence}
                            onClick={() =>
                              download(
                                'puf-design.fasta',
                                `>${
                                  result.targetId
                                }_${
                                  c.id
                                } scaffold=${
                                  plan.scaffold?.id
                                } designed_sequence\n${
                                  plan
                                    .sequence!.match(/.{1,70}/g)
                                    ?.join('\n')
                                }\n`,
                              )
                            }
                          >
                            <Download size={14} />
                            FASTA
                          </Button>
                        </div>
                        {plan.sequence ? (
                          <details>
```





```
                        <summary>{plan.sequence.length} aa</summary>
                        <code>{plan.sequence}</code>
                      </details>
                    ) : (
                      <div className="protein-empty">
                        <Dna size={24} />
                        <p>
                          {plan.scaffold ? 'Missing TRMs.' : 'Scaffold required.'}
                        </p>
                        <button
                          className="text-button"
                          onClick={() => goToStep('optimize')}
                        >
                          Edit PUF <ArrowRight size={13} />
                        </button>
                      </div>
                    )}
                  </div>
                  <div className="panel-heading">
                    <div className="section-title">
                      <Target size={17} />
                      <h3>Bystander C sites</h3>
                    </div>
                    <Pill tone="orange">Binding end +2–5</Pill>
                  </div>
                  <div className="bystanders">
                    {bystanders.length ? (
                      bystanders.map((b) => (
                        <div key={b.position}>
                          <span>C{b.position}</span>
                          <b>+{b.offset} nt</b>
                        </div>
                      ))
                    ) : (
                      <p>None.</p>
                    )}
                  </div>
                  <details className="report-details">
                    <summary>Other formats</summary>
                    <div className="report-actions">
                      <Button variant="outline" onClick={exportReport}>
                        <FileText size={15} />
                        Local report · HTML
                      </Button>
                      <Button variant="outline" onClick={exportRawReport}>
                        <Download size={15} />
                        JSON
                      </Button>
                      <Button variant="outline" onClick={exportHits}>
                        <Download size={15} />
                        Matches · TSV
                      </Button>
                      <Button variant="outline" onClick={() => window.print()}>
                        <FileText size={15} />
                        Print
                      </Button>
                    </div>
                  </details>
                </section>
                <LocalDownloads />
                <div className="step-actions">
```



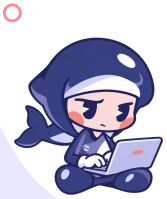


```
                        <Button variant="outline" onClick={() => navigate('optimize')}>
                          Back · PUF
                        </Button>
                      </div>
                    </TabsContent>
                  </Tabs>
                </section>
              </div>
            </>
          )}
          <footer>
            <span>{mode === 'overview' ? 'Programmable RNA editing.' : 'ORCA'}</span>
            <a
              href="https://github.com/pdx12320/igem_software"
              target="_blank"
              rel="noreferrer"
            >
              {mode === 'overview' ? 'Project repository' : 'GitHub'}{' '}
              <ArrowUpRight size={14} />
            </a>
          </footer>
        </main>
        <Dialog open={dataOpen} onOpenChange={setDataOpen}>
          <DialogContent className="data-dialog">
            <DialogTitle>Data</DialogTitle>
            <DialogDescription>TRMs and scaffolds.</DialogDescription>
            {error && (
              <div className="notice error" role="alert">
                {error}
              </div>
            )}
            {message && (
              <div className="notice" role="status">
                {message}
              </div>
            )}
            <div className="data-item">
              <div>
                <Layers />
                <h3>TRMs & scaffolds</h3>
                <Pill>Built in</Pill>
              </div>
              <p>TRMs: 8–12R · scaffolds: 8R / 12R</p>
              <button
                className="text-button"
                onClick={() => {
                  setDataOpen(false);
                  setMode('methods');
                }}
              >
                Methods <ArrowUpRight size={13} />
              </button>
              <p>Library · JSON</p>
              <Input
                aria-label="Upload custom module library"
                type="file"
                accept=".json"
                onChange={(e) => loadLibrary(e.target.files?.[0])}
              />
              <button
                className="text-button"
```





```
                onClick={() => {
                  setLibrary(builtIn);
                  setOverrides({});
                  setMessage('Library reset.');
                }}
              >
                Reset library
              </button>
            </div>
            <Button onClick={() => setDataOpen(false)}>
              Close <ArrowRight size={14} />
            </Button>
          </DialogContent>
        </Dialog>
      </div>
    </IntroExperience>
  );
}
```





components/lab-workspace.tsx

```tsx
'use client';
import { useEffect, useState } from 'react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Textarea } from '@/components/ui/textarea';
import { NativeSelect } from '@/components/ui/native-select';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@/components/ui/tabs';
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from '@/components/ui/table';
import StructureShowcase from '@/components/structure-showcase';
import { checkStructure } from '@/lib/structure-input';
import constructs from '@/data/constructs.json';
import audit from '@/data/structure-showcase.json';
import { assembleFusion, summarize, validateAssays, type LabState } from '@/lib/project';
import type { PredictionSet } from '@/lib/design';
function save(name: string, value: string, type = 'application/json') {
  const u = URL.createObjectURL(new Blob([value], { type }));
  const a = document.createElement('a');
  a.href = u;
  a.download = name;
  a.click();
  setTimeout(() => URL.revokeObjectURL(u), 1000);
}
const sites = ['C388', 'C295', 'C871'] as const;
export default function LabWorkspace({
  state,
  onChange,
  protein,
}: {
  state: LabState;
  onChange: (s: LabState) => void;
  protein: string | null;
  jobTarget?: { id: string; sequence: string; position: number; motif: string };
  onPredictions?: (p: PredictionSet) => void;
}) {
  const [variant, setVariant] = useState(constructs[1].id),
    [assay, setAssay] = useState(''),
    [error, setError] = useState(''),
    [structureOpen, setStructureOpen] = useState(false);
  const [structureChoice, setStructureChoice] = useState('wt'),
    [structureUrl, setStructureUrl] = useState('');
  const candidate =
    variant === 'current-design'
      ? { id: variant, sequence: protein }
      : constructs.find((x) => x.id === variant)!;
  useEffect(() => {
    const text = state.structures?.[variant];
    if (!text) {
      setStructureUrl('');
      return;
```





```
    }
    const u = URL.createObjectURL(new Blob([text], { type: 'text/plain' }));
    setStructureUrl(u);
    return () => URL.revokeObjectURL(u);
  }, [state.structures, variant]);
  async function importStructure(file?: File) {
    if (!file || !candidate.sequence) return;
    try {
      if (file.size > 2000000) throw Error('Structure limit: 2 MB.');
      if (variant === 'current-design')
        throw Error('Choose a named construct for structure import.');
      const cif = checkStructure(await file.text(), candidate.sequence);
      const structures = { ...state.structures, [variant]: cif };
      if (Object.values(structures).reduce((n, s) => n + s.length, 0) > 10000000)
        throw Error('Project structure limit: 10 MB.');
      onChange({ ...state, structures });
      setStructureChoice('variant');
      setStructureOpen(true);
      setError('');
    } catch (e) {
      setError((e as Error).message);
    }
  }
  const wt = constructs[0],
    assays = [...new Set(state.assays.map((r) => r.assay))],
    chosenAssay = assays.includes(assay) ? assay : assays[0] || '';
  const matched = constructs.find((c) => c.sequence === candidate?.sequence);
  const assayId = matched?.id || candidate.id;
  const stats = (id: string, site: string) =>
    summarize(
      state.assays
        .filter((r) => r.construct === id && r.site === site && r.assay === chosenAssay)
        .map((r) => r.editingPercent),
    );
  let fusion: ReturnType<typeof assembleFusion> | null = null;
  let fusionError = '';
  try {
    fusion = assembleFusion(protein, state.fusion);
  } catch (e) {
    fusionError = (e as Error).message;
  }
  async function importAssay(file?: File) {
    if (!file) return;
    try {
      if (file.size > 2000000) throw Error('Experiment file limit: 2 MB.');
      const rows = validateAssays(JSON.parse(await file.text()));
      const unknown = rows.find((r) => !constructs.some((c) => c.id === r.construct));
      if (unknown)
        throw Error(
          'Unknown construct ID: ' +
            unknown.construct +
            '. Use IDs from the construct catalog.',
        );
      onChange({ ...state, assays: rows });
      setError('');
    } catch (e) {
      setError((e as Error).message);
    }
  }
  return (
    <section className="lab-page panel">
```





```
      <div className="panel-heading">
        <h4>Experiments</h4>
      </div>
      <Tabs defaultValue="compare">
        <TabsList
          className="lab-tabs"
          style={{ gridTemplateColumns: 'repeat(3,minmax(0,1fr))' }}
        >
          <TabsTrigger value="compare">Compare</TabsTrigger>
          <TabsTrigger value="validation">Validation</TabsTrigger>
          <TabsTrigger value="fusion">Fusion</TabsTrigger>
        </TabsList>
        {error && (
          <p className="notice error" role="alert">
            {error}
          </p>
        )}
        <TabsContent value="compare">
          <div className="lab-toolbar">
            <label>
              Compare with WT
              <NativeSelect
                value={variant}
                onChange={(e) => {
                  setVariant(e.target.value);
                  setStructureOpen(false);
                }}
              >
                {constructs.slice(1).map((c) => (
                  <option key={c.id} value={c.id}>
                    {c.id}
                  </option>
                ))}
                <option value="current-design">Current workbench design</option>
              </NativeSelect>
            </label>
            <label>
              Experiment conditions
              <NativeSelect
                value={chosenAssay}
                onChange={(e) => setAssay(e.target.value)}
                disabled={!assays.length}
              >
                {assays.length ? (
                  assays.map((a) => <option key={a}>{a}</option>)
                ) : (
                  <option value="">Import experiment data</option>
                )}
              </NativeSelect>
            </label>
          </div>
          <div className="comparison-grid">
            {[wt, candidate].map((c, i) => (
              <article key={i}>
                <span>{i ? 'SELECTED CONSTRUCT' : 'BASELINE'}</span>
                <h3>{c.id}</h3>
                <p>{c.sequence ? c.sequence.length + ' aa' : 'Full scaffold required'}</p>
                <details className="lab-sequence">
                  <summary>Protein sequence</summary>
                  <code className="sequence-diff">
                    {c.sequence?.split('').map((aa, j) => (
```





```
                    <span
                      key={j}
                      className={
                        i &&
                        c.sequence?.length === wt.sequence.length &&
                        aa !== wt.sequence[j]
                          ? 'changed-aa'
                          : ''
                      }
                      title={'Residue ' + (j + 1)}
                    >
                      {aa}
                    </span>
                  )) || 'Protein sequence unavailable'}
                </code>
              </details>
            </article>
          ))}
        </div>
        {candidate.sequence?.length === 492 ? (
          <Table>
            <TableHeader>
              <TableRow>
                <TableHead>Repeat</TableHead>
                <TableHead>WT TRM</TableHead>
                <TableHead>Selected TRM</TableHead>
                <TableHead>Residue changes</TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {audit.repeats.map((r) => {
                const code = r.trmPositions1
                  .map((p) => candidate.sequence![p - 1])
                  .join('');
                return (
                  <TableRow key={r.repeat}>
                    <TableCell>R{r.repeat}</TableCell>
                    <TableCell>{r.wtTrm}</TableCell>
                    <TableCell>
                      <b className={code !== r.wtTrm ? 'changed-aa' : ''}>{code}</b>
                    </TableCell>
                    <TableCell>
                      {r.trmPositions1
                        .filter(
                          (p) => wt.sequence[p - 1] !== candidate.sequence![p - 1],
                        )
                        .map((p) => wt.sequence[p - 1] + p + candidate.sequence![p - 1])
                        .join(', ') || 'Unchanged'}
                    </TableCell>
                  </TableRow>
                );
              })}
            </TableBody>
          </Table>
        ) : (
          <p className="lab-note">
            Repeat-level comparison requires the same 492-aa scaffold.
          </p>
        )}
        <div className="panel-heading">
          <h3>Measured editing · mean ± SD</h3>
```





```
            <Button variant="outline" onClick={() => setStructureOpen((o) => !o)}>
              {structureOpen ? 'Hide' : 'View'} WT structure
            </Button>
          </div>
          <div className="assay-cards">
            {sites.map((site) => {
              const a = stats(wt.id, site),
                b = stats(assayId, site);
              return (
                <article key={site}>
                  <h3>
                    {site}
                    {site === 'C388' ? ' · target' : ' · off-target'}
                  </h3>
                  {[
                    [wt.id, a],
                    [assayId, b],
                  ].map(([label, v], i) => {
                    const x = v as ReturnType<typeof summarize>;
                    return (
                      <div key={i}>
                        <span>{label as string}</span>
                        <b>
                          {x
                            ? x.mean.toFixed(1) +
                              (x.sd === null ? '' : ' ± ' + x.sd.toFixed(1)) +
                              '%'
                            : 'Not supplied'}
                        </b>
                        {x && (
                          <>
                            <meter min="0" max="100" value={x.mean} />
                            <small>
                              n = {x.n}
                              {x.sd === null ? ' · SD unavailable' : ''}
                            </small>
                          </>
                        )}
                      </div>
                    );
                  })}
                  <p>Δ vs WT: {a && b ? (b.mean - a.mean).toFixed(1) + ' pp' : '–'}</p>
                </article>
              );
            })}
          </div>
          <div className="lab-toolbar">
            <label>
              Selected construct · AF3 CIF
              <Input
                type="file"
                accept=".cif"
                disabled={variant === 'current-design'}
                onChange={(e) => {
                  void importStructure(e.target.files?.[0]);
                  e.target.value = '';
                }}
              />
            </label>
            <NativeSelect
              aria-label="Structure to display"
```





```
              value={structureChoice}
              onChange={(e) => {
                setStructureChoice(e.target.value);
                setStructureOpen(true);
              }}
            >
              <option value="wt">WT structure</option>
              <option value="variant" disabled={!structureUrl}>
                Selected construct structure
              </option>
            </NativeSelect>
          </div>
          {structureOpen && (
            <div className="lab-structure">
              {structureChoice === 'variant' && structureUrl ? (
                <StructureShowcase
                  key={structureUrl}
                  modelUrl={structureUrl}
                  modelProtein={candidate.sequence!}
                  modelName={candidate.id + ' · C388 RNA'}
                  external
                />
              ) : (
                <StructureShowcase />
              )}
            </div>
          )}
          <p className="lab-note">
            Required: selected protein on chain A; 15-nt C388 RNA on chain B.
          </p>
          <Button
            variant="outline"
            onClick={() =>
              save(
                'construct-comparison.json',
                JSON.stringify(
                  {
                    baseline: wt,
                    candidate,
                    assay: chosenAssay,
                    measurements: state.assays.filter(
                      (r) =>
                        r.assay === chosenAssay && [wt.id, assayId].includes(r.construct),
                    ),
                  },
                  null,
                  2,
                ),
              )
            }
          >
            Download comparison
          </Button>
        </TabsContent>
        <TabsContent value="validation">
          <div className="lab-toolbar">
            <label>
              Import measured editing · JSON
              <Input
                type="file"
                accept=".json"
```





```
                onChange={(e) => {
                  void importAssay(e.target.files?.[0]);
                  e.target.value = '';
                }}
              />
            </label>
            <Button
              variant="outline"
              onClick={() =>
                save(
                  'experiment-template.json',
                  JSON.stringify(
                    [
                      {
                        construct: wt.id,
                        site: 'C388',
                        assay: 'REPLACE_WITH_CELL_DOSE_TIME_BATCH',
                        replicate: 'bio-1',
                        editingPercent: null,
                      },
                    ],
                    null,
                    2,
                  ),
                )
              }
            >
              Download template
            </Button>
            <Button
              variant="outline"
              onClick={() =>
                save('construct-catalog.json', JSON.stringify(constructs, null, 2))
              }
            >
              Construct IDs & sequences
            </Button>
          </div>
          <details>
            <summary>Import format</summary>
            <p className="lab-note">
              Biological replicates: editing percentages (0–100). Replace template nulls
              with measurements. Include cell type, dose, time and batch in the assay
              label; comparisons use only the selected assay.
            </p>
          </details>
          <p className="lab-note">
            {state.assays.length} records · {assays.length} assay groups
          </p>
          <label className="lab-label">
            Notes
            <Textarea
              value={state.notes}
              maxLength={20000}
              onChange={(e) => onChange({ ...state, notes: e.target.value })}
              placeholder="Changes, results and next steps"
            />
          </label>
          <Table>
            <TableHeader>
              <TableRow>
```





```
                {['Construct', 'Site', 'Assay', 'Replicate', 'Editing (%)'].map((h) => (
                  <TableHead key={h}>{h}</TableHead>
                ))}
              </TableRow>
            </TableHeader>
            <TableBody>
              {state.assays.slice(0, 100).map((r, i) => (
                <TableRow key={i}>
                  <TableCell>{r.construct}</TableCell>
                  <TableCell>{r.site}</TableCell>
                  <TableCell>{r.assay}</TableCell>
                  <TableCell>{r.replicate}</TableCell>
                  <TableCell>{r.editingPercent}</TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
          <p className="lab-note">
            Showing {Math.min(100, state.assays.length)} of {state.assays.length} records.
            Save the project to retain all records and notes.
          </p>
        </TabsContent>
        <TabsContent value="fusion">
          <div className="lab-toolbar">
            <label>
              Fusion order
              <NativeSelect
                value={state.fusion.order}
                onChange={(e) =>
                  onChange({
                    ...state,
                    fusion: {
                      ...state.fusion,
                      order: e.target.value as LabState['fusion']['order'],
                    },
                  })
                }
              >
                <option>PUF-linker-APOBEC</option>
                <option>APOBEC-linker-PUF</option>
              </NativeSelect>
            </label>
            <span>PUF: current design · {protein?.length ?? 0} aa</span>
          </div>
          <div className="fusion-inputs">
            <label>
              Linker sequence
              <Textarea
                value={state.fusion.linker}
                onChange={(e) =>
                  onChange({
                    ...state,
                    fusion: { ...state.fusion, linker: e.target.value },
                  })
                }
                placeholder="Actual linker amino-acid sequence"
              />
            </label>
            <label>
              APOBEC sequence
              <Textarea
```



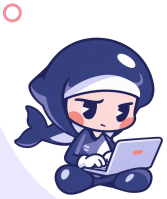


```
              value={state.fusion.apobec}
              onChange={(e) =>
                onChange({
                  ...state,
                  fusion: { ...state.fusion, apobec: e.target.value },
                })
              }
              placeholder="Exact APOBEC variant amino-acid sequence"
            />
          </label>
        </div>
        {fusion ? (
          <>
            <div className="domain-strip">
              {fusion.domains.map((d) => (
                <div
                  key={d.name}
                  className={'domain-' + d.name.toLowerCase()}
                  style={{ flex: Math.max(1, d.sequence.length / 40) }}
                >
                  <b>{d.name}</b>
                  <span>
                    {d.start}–{d.end} · {d.sequence.length} aa
                  </span>
                </div>
              ))}
            </div>
            <code className="fusion-sequence">{fusion.sequence}</code>
            <div className="lab-toolbar">
              <Button
                onClick={() =>
                  save(
                    'puf-apobec-fusion.fasta',
                    '>' +
                      state.fusion.order +
                      ' length=' +
                      fusion!.sequence.length +
                      '\n' +
                      fusion!.sequence.match(/.{1,70}/g)!.join('\n') +
                      '\n',
                    'text/plain',
                  )
                }
              >
                Download fusion FASTA
              </Button>
              <Button
                variant="outline"
                onClick={() =>
                  save('fusion-domains.json', JSON.stringify(fusion, null, 2))
                }
              >
                Domain annotation
              </Button>
            </div>
          </>
```





```
            ) : (
              <p className="lab-note">{fusionError}</p>
            )}
          </TabsContent>
        </Tabs>
      </section>
    );
}
```





components/puf-ranking.tsx

```tsx
'use client';
import { useEffect, useRef, useState } from 'react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from '@/components/ui/table';
import {
  mergeRankCandidates,
  normalizeRankCandidates,
  parseCandidateFasta,
  validateRankCandidates,
  validateRankResult,
  type RankCandidate,
  type RankResult,
} from '@/lib/puf-ranking';

export type PufRankingProps = {
  protein: string | null;
  disabled: boolean;
  serviceReady?: boolean;
  candidates: RankCandidate[];
  onCandidates: (value: RankCandidate[]) => void;
  result: RankResult | null;
  onResult: (value: RankResult | null) => void;
};

export default function PufRanking({
  protein,
  disabled,
  candidates,
  onCandidates,
  result,
  onResult,
}: PufRankingProps) {
  const [name, setName] = useState('');
  const [message, setMessage] = useState('');
  const [operation, setOperation] = useState<{ key: string; token: number } | null>(null);
  const generation = useRef(0);
  const queueKey = JSON.stringify(candidates);
  const contextKey = JSON.stringify([queueKey, disabled]);
  const previousQueue = useRef(queueKey);
  const busy = operation?.key === contextKey;

  useEffect(() => {
    if (previousQueue.current !== queueKey) {
      previousQueue.current = queueKey;
      // Project restoration can replace the queue and its validated result together.
      if (result) {
        try {
          validateRankResult(result, candidates, result.source);
```





```
          return;
        } catch {
          /* Clear unmatched evidence below. */
        }
      }
      onResult(null);
    }
  }, [queueKey, onResult, candidates, result]);

  useEffect(
    () => () => {
      generation.current += 1;
    },
    [contextKey],
  );

  function cancel() {
    generation.current += 1;
    setOperation(null);
  }

  function replaceQueue(value: RankCandidate[]) {
    cancel();
    onResult(null);
    onCandidates(validateRankCandidates(value));
  }

  function enqueueCurrent() {
    try {
      if (disabled || !protein) throw Error('Apply the current design first.');
      let index = 1;
      while (candidates.some((candidate) => candidate.id === 'candidate-' + index))
        index += 1;
      const id = 'candidate-' + index;
      const merged = mergeRankCandidates(
        candidates,
        normalizeRankCandidates([{ id, label: name.trim() || id, protein }]),
      );
      if (merged.length === candidates.length) {
        setMessage('Protein already added.');
        return;
      }
      replaceQueue(merged);
      setName('');
      setMessage('Candidate saved.');
    } catch (error) {
      setMessage((error as Error).message);
    }
  }

  async function importFile(file: File | undefined) {
    if (!file || disabled || busy) return;
    cancel();
    const token = generation.current;
    const snapshot = validateRankCandidates(candidates);
    setOperation({ key: contextKey, token });
    setMessage('');
    try {
      if (file.size > 200_000) throw Error('Import limit: 200 KB.');
      const text = await file.text();
      if (token !== generation.current) return;
```





```
      const merged = mergeRankCandidates(snapshot, parseCandidateFasta(text));
      const added = merged.length - snapshot.length;
      if (added) replaceQueue(merged);
      setMessage(added + ' added. Duplicates skipped.');
    } catch (error) {
      if (token === generation.current) setMessage((error as Error).message);
    } finally {
      if (token === generation.current) setOperation(null);
    }
  }

  return (
    <section className="puf-ranking" aria-labelledby="puf-ranking-title">
      <div className="panel-heading">
        <h3 id="puf-ranking-title">PUF candidates</h3>
      </div>
      <div className="lab-toolbar">
        <label>
          Name
          <Input
            value={name}
            maxLength={200}
            disabled={disabled || busy}
            onChange={(event) => setName(event.target.value)}
            placeholder="Candidate name"
          />
        </label>
        <Button
          type="button"
          disabled={disabled || busy || !protein || candidates.length >= 50}
          onClick={enqueueCurrent}
        >
          Save candidate
        </Button>
        <label>
          Protein FASTA
          <Input
            type="file"
            accept=".fa,.faa,.fasta,.txt"
            disabled={disabled || busy}
            onChange={(event) => {
              void importFile(event.target.files?.[0]);
              event.target.value = '';
            }}
          />
        </label>
      </div>
      {disabled && <p className="notice">Apply RNA changes first.</p>}
      {busy && <p role="status">Processing…</p>}
      {message && (
        <p className="notice" role="status">
          {message}
        </p>
      )}
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead>Candidate</TableHead>
            <TableHead>Length</TableHead>
            <TableHead>
              <span className="sr-only">Queue actions</span>
```





```
            </TableHead>
          </TableRow>
        </TableHeader>
        <TableBody>
          {candidates.map((candidate) => (
            <TableRow key={candidate.id}>
              <TableCell>
                <b>{candidate.label}</b>
              </TableCell>
              <TableCell>{candidate.protein.length} aa</TableCell>
              <TableCell>
                <Button
                  type="button"
                  variant="ghost"
                  size="sm"
                  disabled={disabled}
                  aria-label={'Remove ' + candidate.label + ' from candidate queue'}
                  onClick={() => {
                    replaceQueue(candidates.filter((item) => item.id !== candidate.id));
                    setMessage('Removed.');
                  }}
                >
                  Remove
                </Button>
              </TableCell>
            </TableRow>
          ))}
        </TableBody>
      </Table>
    </section>
  );
}
```





components/offtarget-workspace.tsx

```tsx
'use client';
import { useEffect, useLayoutEffect, useRef, useState } from 'react';
import { Check, Download, FileUp, Loader2, RefreshCw, ScanLine } from 'lucide-react';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import {
  Table,
  TableHeader,
  TableBody,
  TableRow,
  TableHead,
  TableCell,
} from '@/components/ui/table';
import { validateReferenceSites } from '@/lib/reference-sites';
import {
  MAX_FASTA_BYTES,
  OFFTARGET_MODEL_ID,
  targetKey,
  validateJobReceipt,
  validateOffTargetResult,
  resultMatchesInputs,
  type WorkflowTarget,
  type OffTargetState,
  type OffTargetRequest,
} from '@/lib/offtarget-workflow';

function saveJson(name: string, value: unknown) {
  const url = URL.createObjectURL(
    new Blob([JSON.stringify(value, null, 2)], { type: 'application/json' }),
  );
  const a = document.createElement('a');
  a.href = url;
  a.download = name;
  a.click();
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}
export default function OffTargetWorkspace({
  target,
  disabled,
  serviceReady,
  state,
  onChange,
  onPreview,
}: {
  target: WorkflowTarget;
  disabled: boolean;
  serviceReady: boolean;
  state: OffTargetState;
  onChange: (value: OffTargetState) => void;
  onPreview: (file: File) => Promise<void>;
}) {
  const [busy, setBusy] = useState(''),
    [error, setError] = useState(''),
    [message, setMessage] = useState(''),
    [query, setQuery] = useState('');
  const epoch = useRef(0),
```





```
    controller = useRef<AbortController | null>(null),
    latest = useRef(state),
    currentTarget = useRef(targetKey(target));
  useLayoutEffect(() => {
    latest.current = state;
    currentTarget.current = targetKey(target);
  }, [state, target]);
  useEffect(
    () => () => {
      epoch.current++;
      controller.current?.abort();
    },
    [],
  );
  const active = resultMatchesInputs(state, target) ? state.result : null;
  const scopedRequest =
    state.request &&
    targetKey(state.request.target) === targetKey(target) &&
    state.request.reference.sha256 === state.reference?.sha256 &&
    state.request.reference.species === state.species.trim()
      ? state.request
      : null;
  const job = scopedRequest ? state.job : null;
  async function jsonRequest(path: string, init?: RequestInit) {
    const response = await fetch('/api/workflow' + path, {
      ...init,
      credentials: 'same-origin',
      signal: controller.current?.signal,
    });
    const body = await response.json().catch(() => null);
    if (!response.ok)
      throw Error(
        typeof body?.error === 'string'
          ? body.error
          : `Model service returned HTTP ${response.status}.`,
      );
    return body;
  }
  function invalidate(next: Partial<OffTargetState>) {
    epoch.current++;
    controller.current?.abort();
    setError('');
    setMessage('');
    onChange({ ...state, ...next, request: null, job: null, result: null });
  }
  async function chooseFile(file?: File) {
    if (!file) return;
    const previous = state;
    const operation = ++epoch.current;
    controller.current?.abort();
    controller.current = new AbortController();
    setBusy('Checking FASTA…');
    setError('');
    setMessage('');
    onChange({
      ...state,
      file: null,
      reference: null,
      request: null,
      job: null,
      result: null,
```





```
    });
    try {
      if (!file.size || file.size > MAX_FASTA_BYTES)
        throw Error('Upload an uncompressed transcript FASTA up to 100 MiB.');
      if (!(await file.slice(0, 4096).text()).trimStart().startsWith('>'))
        throw Error('Expected a FASTA header beginning with >.');
      await validateReferenceSites(file, [], target, controller.current.signal);
      const hash = Array.from(
        new Uint8Array(await crypto.subtle.digest('SHA-256', await file.arrayBuffer())),
        (b) => b.toString(16).padStart(2, '0'),
      ).join('');
      if (operation !== epoch.current) return;
      const retain =
        previous.reference?.sha256 === hash &&
        previous.reference.species === latest.current.species.trim() &&
        previous.request &&
        targetKey(previous.request.target) === targetKey(target);
      if (retain && previous.result) {
        await validateReferenceSites(
          file,
          previous.result.sites,
          target,
          controller.current.signal,
        );
        if (operation !== epoch.current) return;
      }
      onChange({
        ...latest.current,
        file,
        reference:
          retain && previous.reference
            ? previous.reference
            : {
                name: file.name,
                species: latest.current.species.trim(),
                kind: 'transcript',
                sha256: hash,
                sizeBytes: file.size,
              },
        request: retain ? previous.request : null,
        job: retain ? previous.job : null,
        result: retain ? previous.result : null,
      });
      setMessage('FASTA ready.');
    } catch (e) {
      if (operation === epoch.current) setError((e as Error).message);
    } finally {
      if (operation === epoch.current) setBusy('');
    }
  }
  async function submit() {
    if (
      !state.file ||
      !state.reference ||
      !state.species.trim() ||
      disabled ||
      !serviceReady
    )
      return;
    const operation = ++epoch.current,
      key = targetKey(target);
```





```
    controller.current?.abort();
    controller.current = new AbortController();
    setBusy('Submitting…');
    setError('');
    setMessage('');
    const request: OffTargetRequest = {
      schemaVersion: 1,
      requestId: scopedRequest && !job ? scopedRequest.requestId : crypto.randomUUID(),
      modelId: OFFTARGET_MODEL_ID,
      target: { ...target },
      reference: { ...state.reference, species: state.species.trim() },
      scoreType: 'raw_ranking_score',
    };
    onChange({
      ...state,
      reference: request.reference,
      request,
      job: null,
      result: null,
    });
    try {
      const body = new FormData();
      body.append('metadata', JSON.stringify(request));
      body.append('fasta', state.file, state.file.name);
      const receipt = validateJobReceipt(
        await jsonRequest('/jobs', { method: 'POST', body }),
        request,
      );
      if (operation !== epoch.current || key !== currentTarget.current) return;
      onChange({
        ...latest.current,
        reference: request.reference,
        request,
        job: receipt,
        result: null,
      });
      setMessage('Submitted.');
    } catch (e) {
      if (operation === epoch.current) setError((e as Error).message);
    } finally {
      if (operation === epoch.current) setBusy('');
    }
  }
  async function refresh() {
    if (!scopedRequest || !job || !state.file) return;
    const request = scopedRequest,
      operation = ++epoch.current,
      key = targetKey(target);
    controller.current?.abort();
    controller.current = new AbortController();
    setBusy('Checking status…');
    setError('');
    try {
      const receipt = validateJobReceipt(
        await jsonRequest('/jobs/' + encodeURIComponent(job.id)),
        request,
        job.id,
      );
      if (operation !== epoch.current || key !== currentTarget.current) return;
      onChange({ ...latest.current, job: receipt, result: null });
      if (receipt.status === 'succeeded') {
```





```
        const result = validateOffTargetResult(
          await jsonRequest('/jobs/' + encodeURIComponent(job.id) + '/result'),
          request,
          job.id,
        );
        await validateReferenceSites(
          state.file,
          result.sites,
          target,
          controller.current?.signal,
        );
        if (operation !== epoch.current || key !== currentTarget.current) return;
        onChange({ ...latest.current, job: receipt, result });
        setMessage(`${result.sites.length} sites verified.`);
      } else setMessage('Status: ' + receipt.status + '.');
    } catch (e) {
      if (operation === epoch.current) setError((e as Error).message);
    } finally {
      if (operation === epoch.current) setBusy('');
    }
  }
  async function importResult(file?: File) {
    if (!file || !state.file || !state.reference || !state.species.trim()) return;
    const operation = ++epoch.current,
      key = targetKey(target);
    controller.current?.abort();
    controller.current = new AbortController();
    setBusy('Checking result…');
    setError('');
    try {
      if (file.size > 40 * 1024 * 1024) throw Error('Result JSON limit: 40 MiB.');
      const data = JSON.parse(await file.text());
      const request: OffTargetRequest = scopedRequest ?? {
        schemaVersion: 1,
        requestId: typeof data.requestId === 'string' ? data.requestId : '',
        modelId: OFFTARGET_MODEL_ID,
        target: { ...target },
        reference: { ...state.reference, species: state.species.trim() },
        scoreType: 'raw_ranking_score',
      };
      if (!/^[A-Za-z0-9_-]{1,128}$/.test(request.requestId))
        throw Error('Invalid request ID.');
      const result = validateOffTargetResult(data, request, job?.id);
      await validateReferenceSites(
        state.file,
        result.sites,
        target,
        controller.current?.signal,
      );
      if (operation !== epoch.current || key !== currentTarget.current) return;
      onChange({
        ...latest.current,
        reference: request.reference,
        request,
        job: {
          id: result.jobId,
          requestId: request.requestId,
          referenceSha256: request.reference.sha256,
          status: 'succeeded',
        },
        result,
```





```
      });
      setMessage('Results imported.');
    } catch (e) {
      if (operation === epoch.current) setError((e as Error).message);
    } finally {
      if (operation === epoch.current) setBusy('');
    }
  }
  const visible =
    active?.sites
      .filter((s) => s.transcript.toLowerCase().includes(query.toLowerCase()))
      .sort((a, b) => (b.score ?? -Infinity) - (a.score ?? -Infinity)) ?? [];
  return (
    <section className="panel model-workflow-panel">
      <div className="panel-heading">
        <h2>LAMAR</h2>
        <span className={`pill ${serviceReady ? 'green' : 'neutral'}`}>
          {serviceReady ? 'Ready' : 'Offline'}
        </span>
      </div>
      <p className="model-label">RNA-context scores · PUF-independent</p>
      <div className="model-workflow-body">
        <div className="workflow-input-grid">
          <label htmlFor="reference-species">
            Species
            <Input
              id="reference-species"
              placeholder="Homo sapiens"
              maxLength={200}
              value={state.species}
              disabled={!!busy || disabled}
              onChange={(e) =>
                invalidate({
                  species: e.target.value,
                  reference: state.reference
                    ? { ...state.reference, species: e.target.value.trim() }
                    : null,
                })
              }
            />
          </label>
          <label htmlFor="species-fasta">
            Transcript FASTA · ≤100 MiB
            <Input
              id="species-fasta"
              type="file"
              accept=".fa,.fasta,.fna,.fas,.txt"
              disabled={!!busy || disabled}
              onChange={(e) => {
                void chooseFile(e.target.files?.[0]);
                e.target.value = '';
              }}
            />
          </label>
        </div>
        <p className="model-label">
          mRNA / cDNA · RNA orientation · Unique transcript IDs
        </p>
        {state.reference && !state.file && (
          <p className="notice">Reselect the same FASTA to verify saved results.</p>
        )}
```





```
        {state.reference && (
          <div className="reference-receipt">
            <FileUp size={20} />
            <div>
              <b>{state.reference.name}</b>
              <span>{(state.reference.sizeBytes / 1024).toFixed(1)} KiB</span>
            </div>
            <Check size={18} />
          </div>
        )}
        <div className="builder-actions">
          <Button
            disabled={
              disabled || !!busy || !serviceReady || !state.file || !state.species.trim()
            }
            onClick={() => void submit()}
          >
            {busy ? <Loader2 size={16} className="spin" /> : <ScanLine size={16} />}
            {scopedRequest && !job ? 'Resume' : 'Run LAMAR'}
          </Button>
          <Button
            variant="outline"
            disabled={disabled || !!busy || !state.file || state.file.size > 100000}
            onClick={async () => {
              if (!state.file) return;
              setError('');
              setBusy('Reading subset…');
              try {
                await onPreview(state.file);
                setMessage('Subset applied.');
              } catch (e) {
                setError((e as Error).message);
              } finally {
                setBusy('');
              }
            }}
          >
            Preview · ≤25,000 nt
          </Button>
        </div>
        {busy && <p role="status">{busy}</p>}
        {error && (
          <p className="notice error" role="alert">
            {error}
          </p>
        )}
        {message && (
          <p className="notice" role="status">
            {message}
          </p>
        )}
        {job && (
          <div className="workflow-job">
            <div>
              <span
                className={`pill ${job.status === 'succeeded' ? 'green' : 'neutral'}`}
              >
                {job.status}
              </span>
              {job.message && <p>{job.message}</p>}
            </div>
```





```
            <Button
              variant="outline"
              disabled={!!busy || disabled || !serviceReady || !state.file}
              onClick={() => void refresh()}
            >
              <RefreshCw size={15} />
              Refresh
            </Button>
          </div>
        )}
        <details className="workflow-import">
          <summary>Details & import</summary>
          <p className="model-label">
            Run uploads the RNA, species and FASTA to the model service. Preview is a
            local sequence search.
          </p>
          <p className="model-label">
            Genomic FASTA needs transcript conversion. Cross-species accuracy is
            unvalidated. Raw scores are not editing probabilities.
          </p>
          {state.reference && (
            <p className="model-label">
              Reference SHA-256
              <br />
              <code>{state.reference.sha256}</code>
            </p>
          )}
          {job && (
            <p className="model-label">
              Job: <code>{job.id}</code>
            </p>
          )}
          {active && <p className="model-label">Model: {active.modelVersion}</p>}
          <label htmlFor="lamar-workflow-result">
            Import result JSON
            <Input
              id="lamar-workflow-result"
              type="file"
              accept=".json"
              disabled={
                !!busy ||
                disabled ||
                !state.file ||
                !state.reference ||
                !state.species.trim()
              }
              onChange={(e) => {
                void importResult(e.target.files?.[0]);
                e.target.value = '';
              }}
            />
          </label>
          <div className="builder-actions">
            {scopedRequest && (
              <Button
                variant="outline"
                onClick={() => saveJson('ORCA-LAMAR-request.json', scopedRequest)}
              >
                <Download size={14} />
                Save request
              </Button>
```



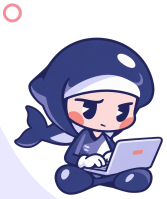


```
            )}
            {active && (
              <Button
                variant="outline"
                onClick={() => saveJson('ORCA-LAMAR-result.json', active)}
              >
                <Download size={14} />
                Save result
              </Button>
            )}
          </div>
        </details>
        {active && (
          <>
            <div className="workflow-result-heading">
              <div>
                <h3>Sites</h3>
                <p>
                  {active.sites.length} total ·{' '}
                  {active.sites.filter((s) => s.score !== null).length} scored
                </p>
              </div>
              <Input
                aria-label="Search model site transcript IDs"
                placeholder="Find transcript…"
                value={query}
                onChange={(e) => setQuery(e.target.value)}
              />
            </div>
            <Table>
              <TableHeader>
                <TableRow>
                  <TableHead>Transcript</TableHead>
                  <TableHead>C position · 1-based</TableHead>
                  <TableHead>Raw score</TableHead>
                  <TableHead>Status</TableHead>
                </TableRow>
              </TableHeader>
              <TableBody>
                {visible.slice(0, 100).map((s, i) => (
                  <TableRow key={i}>
                    <TableCell>
                      {s.transcript}
                      {s.transcript === target.id && s.position === target.position && (
                        <span className="pill neutral">Target</span>
                      )}
                    </TableCell>
                    <TableCell>{s.position}</TableCell>
                    <TableCell>{s.score === null ? '–' : s.score.toFixed(4)}</TableCell>
                    <TableCell>{s.status.replaceAll('_', ' ')}</TableCell>
                  </TableRow>
                ))}
              </TableBody>
            </Table>
            <p className="model-label">
              {Math.min(100, visible.length)} / {visible.length} shown · CSV includes all
              sites. Missing scores do not mean zero risk.
            </p>
            {!active.sites.length && (
              <p className="notice">
                No sites returned; off-target editing is not ruled out.
```





```
545                 </p>
546               )}
547             </>
548           )}
549         </div>
550       </section>
551     );
552   }
```





components/local-downloads.tsx

```tsx
import { Download, FileText, ShieldCheck } from 'lucide-react';
import { Button } from '@/components/ui/button';
import release from '@/data/local-installer-release.json';

function WindowsMark() {
  return (
    <svg viewBox="0 0 32 32" aria-hidden="true" focusable="false">
      <path
        fill="currentColor"
        d="M3 3h12v12H3zm14 0h12v12H17zM3 17h12v12H3zm14 0h12v12H17z"
      />
    </svg>
  );
}

function MacMark() {
  return (
    <svg viewBox="0 0 36 40" aria-hidden="true" focusable="false">
      <path
        fill="currentColor"
        d="M24 1c.2 2.6-.8 5.1-2.4 6.8-1.7 1.8-4.2 2.8-6.5 2.6-.2-2.4.9-5 2.5-6.6C19.3 2 21.8 1 24
 1Zm-.4 10c-2.5-.1-4.7 1.5-6.1 1.5-1.5 0-3.6-1.4-5.8-1.4-3 0-5.8 1.7-7.2 4.4-3.1 5.6-.8 13.9 2.3 1
8.1 1.6 2.1 3.3 4.4 5.7 4.3 2.3-.1 3.2-1.5 5.8-1.5 2.8 0 3.6 1.5 6.1 1.4 2.5-.1 4-2.1 5.5-4.4 1.6-
2.4 2.3-4.7 2.4-4.8-.1 0-4.8-1.8-4.8-7-.1-4.4 3.6-6.4 3.8-6.6-2.1-3-5.3-3.5-6.5-3.6Z"
      />
    </svg>
  );
}

const platforms = [
  {
    id: 'windows' as const,
    name: 'Windows',
    subtitle: 'Windows x64',
    action: 'Download for Windows',
    Mark: WindowsMark,
  },
  {
    id: 'macos' as const,
    name: 'macOS',
    subtitle: 'Apple Silicon · Intel',
    action: 'Download for macOS',
    Mark: MacMark,
  },
];

export default function LocalDownloads() {
  return (
    <section className="app-downloads" aria-labelledby="local-download-title">
      <div className="app-downloads-heading">
        <h2 id="local-download-title">ORCA for your computer</h2>
      </div>
      <div className="app-download-grid">
        {platforms.map(({ id, name, subtitle, action, Mark }) => (
          <article
            className="app-download-card"
```





```
            key={id}
            aria-labelledby={'download-' + id + '-title'}
          >
            <div className="app-download-card-top">
              <span className={'app-platform-mark ' + id}>
                <Mark />
              </span>
            </div>
            <div className="app-download-card-copy">
              <h3 id={'download-' + id + '-title'}>{name}</h3>
              <p className="app-download-platform">{subtitle}</p>
            </div>
            <div className="app-download-card-bottom">
              <Button asChild className="app-download-action">
                <a href={release.urls[id]} download={release.fileName}>
                  <Download size={17} aria-hidden="true" />
                  {action}
                </a>
              </Button>
            </div>
          </article>
        ))}
      </div>
      <details className="app-download-help">
        <summary>Installation</summary>
        <div className="app-download-help-body">
          <p>
            Extract the ZIP and open the launcher for your system. First launch installs
            the models. Enter your website CSV and species FASTA paths to run locally.
          </p>
          <div className="app-download-help-links">
            <a href="/orca-local-v0.2-guide.txt" download>
              <FileText size={15} aria-hidden="true" />
              Setup guide
            </a>
            <a href="/orca-local-v0.2-sha256.txt" download>
              <ShieldCheck size={15} aria-hidden="true" />
              SHA-256
            </a>
          </div>
        </div>
      </details>
    </section>
  );
}
```





lib/project.ts

```ts
import constructs from '../data/constructs.json';
import { validateStructureEvidence, type StructureResult } from './af3-analysis';
import { checkStructure } from './structure-input';
import {
  analyze,
  normalizeRNA,
  parseFasta,
  validateLibrary,
  validatePredictions,
  modulePlan,
  type ModuleLibrary,
  type PredictionSet,
} from './design';
export type AssayRow = {
  construct: string;
  site: 'C388' | 'C295' | 'C871';
  assay: string;
  replicate: string;
  editingPercent: number;
};
export type Fusion = {
  order: 'PUF-linker-APOBEC' | 'APOBEC-linker-PUF';
  linker: string;
  apobec: string;
};
export type LabState = {
  assays: AssayRow[];
  fusion: Fusion;
  notes: string;
  structures?: Record<string, string>;
};
export const emptyLab: LabState = {
  assays: [],
  fusion: { order: 'PUF-linker-APOBEC', linker: '', apobec: '' },
  notes: '',
};
export function validateAssays(value: unknown): AssayRow[] {
  if (!Array.isArray(value) || value.length > 10000)
    throw Error('Expected up to 10,000 experiment rows.');
  const seen = new Set<string>();
  return value.map((r) => {
    if (
      !r ||
      !['construct', 'assay', 'replicate'].every(
        (k) => typeof r[k] === 'string' && r[k].trim() && r[k].length <= 200,
      ) ||
      !['C388', 'C295', 'C871'].includes(r.site) ||
      typeof r.editingPercent !== 'number' ||
      !Number.isFinite(r.editingPercent) ||
      r.editingPercent < 0 ||
      r.editingPercent > 100
    )
      throw Error(
        'Each row needs construct, site, assay, replicate and editingPercent (0–100).',
      );
    const key = JSON.stringify([r.construct, r.site, r.assay, r.replicate]);
```





```
    if (seen.has(key)) throw Error('Duplicate construct/site/assay/replicate.');
    seen.add(key);
    return {
      construct: r.construct,
      site: r.site,
      assay: r.assay,
      replicate: r.replicate,
      editingPercent: r.editingPercent,
    };
  });
}
export function summarize(values: number[]) {
  if (!values.length) return null;
  const mean = values.reduce((a, b) => a + b, 0) / values.length;
  return {
    n: values.length,
    mean,
    sd:
      values.length > 1
        ? Math.sqrt(values.reduce((a, b) => a + (b - mean) ** 2, 0) / (values.length - 1))
        : null,
  };
}
export function proteinSequence(s: string) {
  const out = s.replace(/\s/g, '').toUpperCase();
  if (!/^[ACDEFGHIKLMNPQRSTVWY]+$/.test(out) || out.length > 10000)
    throw Error('Use a protein sequence of 1–10,000 standard amino acids.');
  return out;
}
export function assembleFusion(puf: string | null, f: Fusion) {
  if (!puf)
    throw Error('Select an 8R / 12R scaffold or supply a complete scaffold first.');
  const parts = {
    PUF: proteinSequence(puf),
    linker: proteinSequence(f.linker),
    APOBEC: proteinSequence(f.apobec),
  };
  if (!['PUF-linker-APOBEC', 'APOBEC-linker-PUF'].includes(f.order))
    throw Error('Invalid fusion order.');
  let end = 0;
  const domains = f.order.split('-').map((name) => {
    const seq = parts[name as keyof typeof parts];
    const start = end + 1;
    end += seq.length;
    return { name, start, end, sequence: seq };
  });
  return { sequence: domains.map((d) => d.sequence).join(''), domains };
}
export function validateLab(v: unknown): LabState {
  const x = v as LabState;
  if (
    !x ||
    typeof x.notes !== 'string' ||
    x.notes.length > 20000 ||
    !x.fusion ||
    !['PUF-linker-APOBEC', 'APOBEC-linker-PUF'].includes(x.fusion.order) ||
    typeof x.fusion.linker !== 'string' ||
    typeof x.fusion.apobec !== 'string' ||
    x.fusion.linker.length > 10000 ||
    x.fusion.apobec.length > 10000
  )
```





```
    throw Error('Invalid project lab data.');
  const structures: Record<string, string> = {};
  let size = 0;
  for (const [id, text] of Object.entries(x.structures || {})) {
    const c = constructs.find((c) => c.id === id);
    if (!c || typeof text !== 'string') throw Error('Unknown saved structure.');
    size += text.length;
    if (size > 10000000) throw Error('Saved structures exceed 10 MB.');
    structures[id] = checkStructure(text, c.sequence);
  }
  return {
    assays: validateAssays(x.assays),
    fusion: x.fusion,
    notes: x.notes,
    structures,
  };
}
export type ProjectFile = {
  kind: 'puf-studio-project';
  version: 1;
  target: { id: string; sequence: string; position: number };
  length: number;
  windowStart: number;
  referenceName: string;
  references: { id: string; sequence: string }[];
  library: ModuleLibrary;
  predictions: PredictionSet | null;
  overrides: Record<number, string>;
  lab: LabState;
  structureEvidence?: StructureResult[];
  experience?: 'beginner' | 'expert';
};
export function restoreProject(input: unknown) {
  const p = input as ProjectFile;
  if (!p || p.kind !== 'puf-studio-project' || p.version !== 1)
    throw Error('Upload a PUF Studio project, version 1.');
  if (
    !p.target ||
    typeof p.target.sequence !== 'string' ||
    typeof p.target.id !== 'string' ||
    typeof p.referenceName !== 'string' ||
    !Array.isArray(p.references) ||
    !p.overrides ||
    typeof p.overrides !== 'object'
  )
    throw Error('Incomplete project.');
  const seq = normalizeRNA(p.target.sequence);
  if (
    p.references.some(
      (r) =>
        !r ||
        typeof r.id !== 'string' ||
        typeof r.sequence !== 'string' ||
        /[\s<>]/.test(r.id),
    )
  )
    throw Error('Invalid reference record.');
  const refs = p.references.length
    ? parseFasta(p.references.map((r) => '>' + r.id + '\n' + r.sequence).join('\n'))
    : [];
  const library = validateLibrary(p.library),
```





```
    predictions = p.predictions ? validatePredictions(p.predictions) : null,
    lab = validateLab(p.lab);
  const settings = { recognitionLength: p.length, mismatch: 2 };
  const candidates = analyze(seq, p.target.position, p.target.id, refs, settings);
  const chosen = candidates.find((c) => c.start === p.windowStart);
  if (!chosen) throw Error('Saved window does not match the target.');
  if (p.experience !== undefined && !['beginner', 'expert'].includes(p.experience))
    throw Error('Invalid saved experience mode.');
  if (p.structureEvidence !== undefined) validateStructureEvidence(p.structureEvidence);
  const base = modulePlan(chosen, library);
  for (const [k, v] of Object.entries(p.overrides)) {
    const row = base.rows.find((r) => r.repeat === Number(k));
    if (typeof v !== 'string' || !row?.options.some((m) => m.id === v))
      throw Error('Saved TRM selection is incompatible with its library.');
  }
  return {
    p: {
      ...p,
      target: { ...p.target, sequence: seq },
      references: refs,
      library,
      predictions,
      lab,
    },
    candidates,
    chosen,
    settings,
  };
}
```





lib/workflow-project.ts

```ts
import {
  validateRankCandidates,
  validateRankResult,
  type RankCandidate,
  type RankResult,
} from './puf-ranking';
import {
  emptyOffTargetState,
  MAX_FASTA_BYTES,
  OFFTARGET_MODEL_ID,
  targetKey,
  validateJobReceipt,
  validateOffTargetResult,
  type OffTargetState,
  type OffTargetRequest,
  type UploadedReference,
  type WorkflowTarget,
} from './offtarget-workflow';

export type WorkflowSnapshot = {
  candidates: RankCandidate[];
  ranking: RankResult | null;
  offtarget: OffTargetState;
};
export type SavedWorkflow = {
  schemaVersion: 1;
  candidates: RankCandidate[];
  ranking: RankResult | null;
  offtarget: Omit<OffTargetState, 'file'>;
};

function object(value: unknown): Record<string, unknown> {
  if (!value || typeof value !== 'object' || Array.isArray(value))
    throw Error('Expected a workflow JSON object.');
  return value as Record<string, unknown>;
}
function boundedText(value: unknown, max: number, allowEmpty = false): value is string {
  return (
    typeof value === 'string' &&
    value.length <= max &&
    !/[\x00-\x1f\x7f]/.test(value) &&
    (allowEmpty || !!value.trim())
  );
}
function validateTarget(value: unknown): WorkflowTarget {
  const target = object(value);
  if (
    !boundedText(target.id, 200) ||
    /[\s<>]/.test(target.id) ||
    typeof target.sequence !== 'string' ||
    !/^[ACGU]{1,25000}$/.test(target.sequence) ||
    !Number.isSafeInteger(target.position) ||
    (target.position as number) < 1 ||
    (target.position as number) > target.sequence.length ||
    target.sequence[(target.position as number) - 1] !== 'C' ||
    typeof target.motif !== 'string' ||
```





```
    !/^[ACGU]{8,12}$/.test(target.motif) ||
    !Number.isSafeInteger(target.bindingStart) ||
    (target.bindingStart as number) < 1 ||
    target.sequence.slice(
      (target.bindingStart as number) - 1,
      (target.bindingStart as number) - 1 + target.motif.length,
    ) !== target.motif
  ) {
    throw Error(
      'Saved workflow target needs standard RNA, a 1-based C position and the exact 8–12 nt bindin
g interval.',
    );
  }
  return {
    id: target.id,
    sequence: target.sequence,
    position: target.position as number,
    motif: target.motif,
    bindingStart: target.bindingStart as number,
  };
}
function validateReference(value: unknown): UploadedReference {
  const reference = object(value);
  if (
    !boundedText(reference.name, 512) ||
    !boundedText(reference.species, 200, true) ||
    reference.species !== reference.species.trim() ||
    reference.kind !== 'transcript' ||
    typeof reference.sha256 !== 'string' ||
    !/^[a-f0-9]{64}$/.test(reference.sha256) ||
    !Number.isSafeInteger(reference.sizeBytes) ||
    (reference.sizeBytes as number) < 1 ||
    (reference.sizeBytes as number) > MAX_FASTA_BYTES
  ) {
    throw Error(
      'Saved reference needs its filename, species, transcript type, SHA256 and bounded byte size.
',
    );
  }
  return {
    name: reference.name,
    species: reference.species,
    kind: 'transcript',
    sha256: reference.sha256,
    sizeBytes: reference.sizeBytes as number,
  };
}
function validateRequest(
  value: unknown,
  reference: UploadedReference,
  target?: WorkflowTarget,
): OffTargetRequest {
  const request = object(value),
    savedReference = validateReference(request.reference),
    savedTarget = validateTarget(request.target);
  if (
    request.schemaVersion !== 1 ||
    request.modelId !== OFFTARGET_MODEL_ID ||
    request.scoreType !== 'raw_ranking_score' ||
    typeof request.requestId !== 'string' ||
    !/^[A-Za-z0-9_-]{1,128}$/.test(request.requestId) ||
```





```
    !savedReference.species ||
    JSON.stringify(savedReference) !== JSON.stringify(reference) ||
    (target && targetKey(savedTarget) !== targetKey(target))
  ) {
    throw Error(
      'Saved model request does not match the project target, species, reference or raw-score cont
ract.',
    );
  }
  return {
    schemaVersion: 1,
    requestId: request.requestId,
    modelId: OFFTARGET_MODEL_ID,
    target: savedTarget,
    reference: savedReference,
    scoreType: 'raw_ranking_score',
  };
}

/** Restore data only: browser File objects and running transports never survive a project file. *
/
export function restoreWorkflow(
  input: unknown,
  target?: WorkflowTarget,
): WorkflowSnapshot {
  const currentTarget = target ? validateTarget(target) : undefined;
  if (input === undefined)
    return { candidates: [], ranking: null, offtarget: { ...emptyOffTargetState } };
  const saved = object(input);
  if (saved.schemaVersion !== 1) throw Error('Unsupported workflow project schema.');
  const candidates = validateRankCandidates(saved.candidates);
  let ranking: RankResult | null = null;
  if (saved.ranking !== null) {
    const ranked = object(saved.ranking);
    if (
      ranked.source !== 'bundled' &&
      ranked.source !== 'service' &&
      ranked.source !== 'imported'
    )
      throw Error('Saved ranking requires its result source.');
    ranking = validateRankResult(ranked, candidates, ranked.source);
  }
  const source = object(saved.offtarget);
  if (!boundedText(source.species, 200, true)) throw Error('Invalid saved species name.');
  const species = source.species.trim();
  const reference =
    source.reference === null ? null : validateReference(source.reference);
  if (reference && reference.species !== species)
    throw Error('Saved species and reference metadata disagree.');
  let request: OffTargetRequest | null = null;
  if (source.request !== null) {
    if (!reference) throw Error('Saved model requests require reference metadata.');
    request = validateRequest(source.request, reference, currentTarget);
  }
  if (!request && (source.job !== null || source.result !== null))
    throw Error('Saved job and results require their original model request.');
  const job = source.job === null ? null : validateJobReceipt(source.job, request!);
  if (source.result !== null && (!job || job.status !== 'succeeded'))
    throw Error('Saved results require the matching completed job receipt.');
  const result =
    source.result === null
```





```
      ? null
      : validateOffTargetResult(source.result, request!, job!.id);
  return {
    candidates,
    ranking,
    offtarget: { file: null, reference, species, request, job, result },
  };
}

/** Save immutable evidence, excluding the selected File and evidence for a previous target. */
export function serializeWorkflow(
  input: WorkflowSnapshot,
  target?: WorkflowTarget,
): SavedWorkflow {
  const currentTarget = target ? validateTarget(target) : undefined;
  const stale = !!(
    currentTarget &&
    input.offtarget.request &&
    targetKey(input.offtarget.request.target) !== targetKey(currentTarget)
  );
  const { reference, species } = input.offtarget;
  const candidate: SavedWorkflow = {
    schemaVersion: 1,
    candidates: input.candidates,
    ranking: input.ranking,
    offtarget: {
      reference,
      species,
      request: stale ? null : input.offtarget.request,
      job: stale ? null : input.offtarget.job,
      result: stale ? null : input.offtarget.result,
    },
  };
  const restored = restoreWorkflow(candidate, currentTarget);
  return {
    schemaVersion: 1,
    candidates: restored.candidates,
    ranking: restored.ranking,
    offtarget: {
      reference: restored.offtarget.reference,
      species: restored.offtarget.species,
      request: restored.offtarget.request,
      job: restored.offtarget.job,
      result: restored.offtarget.result,
    },
  };
}
```





lib/puf-ranking.ts

```ts
export const RANK_MODEL_ID = 'module2-fixed492' as const;
export const RANK_REFERENCE_SHA256 =
  '2b981a33277b03de9a6542f27baf6c56626ec3f21c774f017cb5880ac05b32f9';
export const RANK_SOURCE_CHECKPOINT_SHA256 =
  'c0fb39463be5468a2a20739a1838f4ed1a4405003b8914e71eec82984739c34e';
export const RANK_COMPATIBILITY_VERSION = 'drop-fixed-terminal-g-v1' as const;
export const RANK_SCORE_DEFINITION = 'C388_hat - (C295_hat + C871_hat)/2' as const;
export const RANK_SCOPE = 'fixed-assay-C295-C388-C871' as const;
export const MAX_RANK_CANDIDATES = 50;

export type RankCandidate = {
  id: string;
  label: string;
  protein: string;
  normalization?: typeof RANK_COMPATIBILITY_VERSION;
  originalProtein?: string;
};
export type RankedRow = {
  id: string;
  protein: string;
  rank: number;
  C295_hat: number;
  C388_hat: number;
  C871_hat: number;
  S_hat: number;
  ESM2_score: number;
};
export type RankResult = {
  schemaVersion: 1;
  modelId: typeof RANK_MODEL_ID;
  modelVersion: string;
  referenceSha256: string;
  scoreDefinition: typeof RANK_SCORE_DEFINITION;
  sourceCheckpointSha256: string;
  compatibilityVersion: typeof RANK_COMPATIBILITY_VERSION;
  scope: typeof RANK_SCOPE;
  rows: RankedRow[];
  source: 'bundled' | 'service' | 'imported';
};

function record(value: unknown): value is Record<string, unknown> {
  return value !== null && typeof value === 'object' && !Array.isArray(value);
}

/** Validate and copy snapshots; callers cannot mutate the input through the result. */
export function validateRankCandidates(value: unknown): RankCandidate[] {
  if (!Array.isArray(value) || value.length > MAX_RANK_CANDIDATES) {
    throw Error('Keep at most 50 candidate proteins in the queue.');
  }
  const ids = new Set<string>(),
    proteins = new Set<string>();
  return value.map((item) => {
    if (
      !record(item) ||
      typeof item.id !== 'string' ||
      !/^[A-Za-z0-9][A-Za-z0-9_.:-]{0,119}$/.test(item.id)
```





```
    ) {
      throw Error(
        'Candidate IDs must be 1–120 letters, digits, dots, colons, underscores or hyphens, starti
ng with a letter or digit.',
      );
    }
    if (
      typeof item.label !== 'string' ||
      !item.label.trim() ||
      item.label.length > 200 ||
      /[\x00-\x1f\x7f]/.test(item.label)
    ) {
      throw Error('Each candidate needs a name of 1–200 characters.');
    }
    if (
      typeof item.protein !== 'string' ||
      !/^[ACDEFGHIKLMNPQRSTVWY]{1,2000}$/.test(item.protein)
    ) {
      throw Error('Each protein must contain 1–2,000 uppercase standard amino acids.');
    }
    if (
      (item.normalization !== undefined || item.originalProtein !== undefined) &&
      (item.normalization !== RANK_COMPATIBILITY_VERSION ||
        item.protein.length !== 492 ||
        item.originalProtein !== item.protein + 'G')
    ) {
      throw Error(
        'Normalized candidates must preserve their original 493-aa sequence with only terminal G r
emoved.',
      );
    }
    if (ids.has(item.id)) throw Error('Duplicate candidate ID: ' + item.id + '.');
    if (proteins.has(item.protein))
      throw Error('Duplicate protein sequence: ' + item.id + '.');
    ids.add(item.id);
    proteins.add(item.protein);
    return {
      id: item.id,
      label: item.label.trim(),
      protein: item.protein,
      ...(item.normalization === RANK_COMPATIBILITY_VERSION
        ? {
            normalization: RANK_COMPATIBILITY_VERSION,
            originalProtein: item.originalProtein as string,
          }
        : {}),
    };
  });
}

/** User-authorized coordinate normalization; other lengths and terminal variants stay unchanged.
*/
export function normalizeRankCandidates(value: RankCandidate[]): RankCandidate[] {
  const normalized = validateRankCandidates(value).map((candidate) =>
    candidate.protein.length === 493 && candidate.protein.endsWith('G')
      ? {
          ...candidate,
          protein: candidate.protein.slice(0, 492),
          originalProtein: candidate.protein,
          normalization: RANK_COMPATIBILITY_VERSION,
        }
```





```
      : candidate,
  );
  return validateRankCandidates(normalized);
}

export function parseCandidateFasta(text: string): RankCandidate[] {
  if (text.length > 200_000) throw Error('Protein FASTA limit: 200 KB.');
  const candidates: RankCandidate[] = [];
  let current: RankCandidate | null = null;
  for (const raw of text.replace(/^\uFEFF/, '').split(/\r?\n/)) {
    const line = raw.trim();
    if (!line) continue;
    if (line.startsWith('>')) {
      if (current) candidates.push(current);
      const header = line.slice(1).trim(),
        id = header.split(/\s+/)[0];
      current = { id, label: header, protein: '' };
      if (candidates.length >= MAX_RANK_CANDIDATES)
        throw Error('Protein FASTA limit: 50 records.');
    } else {
      if (!current) throw Error('Protein FASTA must start with a >identifier header.');
      current.protein += line.toUpperCase();
      if (current.protein.length > 2000)
        throw Error('Protein FASTA limit: 2,000 amino acids per record.');
    }
  }
  if (current) candidates.push(current);
  if (!candidates.length) throw Error('Protein FASTA contains no records.');
  return normalizeRankCandidates(candidates);
}

/** Preserve existing IDs; skip an already queued sequence without changing it. */
export function mergeRankCandidates(
  existing: RankCandidate[],
  added: RankCandidate[],
): RankCandidate[] {
  const merged = validateRankCandidates(existing);
  for (const item of validateRankCandidates(added)) {
    const sameId = merged.find((candidate) => candidate.id === item.id);
    if (sameId && sameId.protein !== item.protein)
      throw Error(
        'Candidate ID already belongs to a different protein: ' + item.id + '.',
      );
    if (!merged.some((candidate) => candidate.protein === item.protein))
      merged.push({ ...item });
  }
  return validateRankCandidates(merged);
}

/** Validate received measurements without producing predictions or changing scores. */
export function validateRankResult(
  value: unknown,
  candidates: RankCandidate[],
  source: RankResult['source'] = 'imported',
): RankResult {
  const expected = validateRankCandidates(candidates);
  if (
    !expected.length ||
    expected.some((candidate) => candidate.protein.length !== 492)
  ) {
    throw Error(
```





```
      'This result requires a nonempty queue of 492-aa proteins in the fixed reference alignment.'
,
    );
  }
  if (
    !record(value) ||
    value.schemaVersion !== 1 ||
    value.modelId !== RANK_MODEL_ID ||
    typeof value.modelVersion !== 'string' ||
    !value.modelVersion.trim() ||
    value.modelVersion.length > 200 ||
    value.referenceSha256 !== RANK_REFERENCE_SHA256 ||
    value.sourceCheckpointSha256 !== RANK_SOURCE_CHECKPOINT_SHA256 ||
    value.compatibilityVersion !== RANK_COMPATIBILITY_VERSION ||
    value.scoreDefinition !== RANK_SCORE_DEFINITION ||
    value.scope !== RANK_SCOPE ||
    !['bundled', 'service', 'imported'].includes(source)
  ) {
    throw Error(
      'Result provenance must identify schema 1, module2-fixed492, model version, source checkpoin
t, terminal-G compatibility version, reference SHA256 and fixed three-assay score definition.',
    );
  }
  if (!Array.isArray(value.rows) || value.rows.length !== expected.length) {
    throw Error('Results must contain exactly one row for every queued candidate.');
  }
  const expectedById = new Map(expected.map((candidate) => [candidate.id, candidate]));
  const seen = new Set<string>();
  const fields = ['C295_hat', 'C388_hat', 'C871_hat', 'S_hat', 'ESM2_score'] as const;
  const rows = value.rows.map((item) => {
    if (!record(item) || typeof item.id !== 'string' || seen.has(item.id))
      throw Error('Result candidate IDs must be unique.');
    const candidate = expectedById.get(item.id);
    if (!candidate || item.protein !== candidate.protein)
      throw Error(
        'Result candidate ID and complete protein must exactly match the queue.',
      );
    if (
      fields.some(
        (field) => typeof item[field] !== 'number' || !Number.isFinite(item[field]),
      )
    ) {
      throw Error(
        'All received model scores must be finite numbers; missing values cannot be treated as zer
o.',
      );
    }
    const numbers = Object.fromEntries(
      fields.map((field) => [field, item[field]]),
    ) as Pick<RankedRow, (typeof fields)[number]>;
    const formula = numbers.C388_hat - numbers.C295_hat / 2 - numbers.C871_hat / 2;
    const tolerance = 1e-9 * Math.max(1, Math.abs(formula), Math.abs(numbers.S_hat));
    if (!Number.isFinite(formula) || Math.abs(numbers.S_hat - formula) > tolerance) {
      throw Error('Received S_hat does not match C388_hat - (C295_hat + C871_hat)/2.');
    }
    seen.add(item.id);
    return { id: item.id, protein: candidate.protein, ...numbers, rank: 0 };
  });
  // Match archived predict.py: descending S_hat, then ascending candidate ID.
  rows.sort((a, b) => b.S_hat - a.S_hat || (a.id < b.id ? -1 : a.id > b.id ? 1 : 0));
  rows.forEach((row, index) => {
```





```
    row.rank = index + 1;
  });
  return {
    schemaVersion: 1,
    modelId: RANK_MODEL_ID,
    modelVersion: value.modelVersion,
    referenceSha256: RANK_REFERENCE_SHA256,
    scoreDefinition: RANK_SCORE_DEFINITION,
    sourceCheckpointSha256: RANK_SOURCE_CHECKPOINT_SHA256,
    compatibilityVersion: RANK_COMPATIBILITY_VERSION,
    scope: RANK_SCOPE,
    rows,
    source,
  };
}
```





lib/offtarget-workflow.ts

```ts
// Transport and evidence validation only. Biological inference runs in the model service.
export const OFFTARGET_MODEL_ID = 'module4-lamar-101';
export const MAX_FASTA_BYTES = 100 * 1024 * 1024;
export type WorkflowTarget = {
  id: string;
  sequence: string;
  position: number;
  motif: string;
  bindingStart: number;
};
export type UploadedReference = {
  name: string;
  species: string;
  kind: 'transcript';
  sha256: string;
  sizeBytes: number;
};
export type OffTargetRequest = {
  schemaVersion: 1;
  requestId: string;
  modelId: typeof OFFTARGET_MODEL_ID;
  target: WorkflowTarget;
  reference: UploadedReference;
  scoreType: 'raw_ranking_score';
};
export type JobReceipt = {
  id: string;
  requestId: string;
  referenceSha256: string;
  status: 'queued' | 'running' | 'succeeded' | 'failed';
  message?: string;
};
export type OffTargetSite = {
  transcript: string;
  position: number;
  context: string | null;
  score: number | null;
  bindingStart?: number;
  mismatches?: number;
  status: 'scored' | 'unscored' | 'insufficient_context';
};
export type OffTargetResult = {
  schemaVersion: 1;
  jobId: string;
  requestId: string;
  modelId: typeof OFFTARGET_MODEL_ID;
  modelVersion: string;
  referenceSha256: string;
  species: string;
  referenceKind: 'transcript';
  scope: 'uploaded-transcript-fasta';
  scoreType: 'raw_ranking_score';
  target: WorkflowTarget;
  sites: OffTargetSite[];
};
export type OffTargetState = {
```





```
  file: File | null;
  reference: UploadedReference | null;
  species: string;
  request: OffTargetRequest | null;
  job: JobReceipt | null;
  result: OffTargetResult | null;
};
export const emptyOffTargetState: OffTargetState = {
  file: null,
  reference: null,
  species: '',
  request: null,
  job: null,
  result: null,
};
export function targetKey(target: WorkflowTarget) {
  return JSON.stringify([
    target.id,
    target.sequence,
    target.position,
    target.motif,
    target.bindingStart,
  ]);
}
function object(value: unknown): Record<string, unknown> {
  if (!value || typeof value !== 'object' || Array.isArray(value))
    throw Error('Expected a JSON object.');
  return value as Record<string, unknown>;
}
function text(value: unknown, max = 300): value is string {
  return typeof value === 'string' && value.trim().length > 0 && value.length <= max;
}
export function validateJobReceipt(
  value: unknown,
  request: OffTargetRequest,
  expectedId?: string,
): JobReceipt {
  const x = object(value);
  if (
    !text(x.id, 128) ||
    !/^[A-Za-z0-9][A-Za-z0-9_-]{0,127}$/.test(x.id) ||
    x.requestId !== request.requestId ||
    x.referenceSha256 !== request.reference.sha256 ||
    !['queued', 'running', 'succeeded', 'failed'].includes(x.status as string) ||
    (expectedId !== undefined && x.id !== expectedId)
  )
    throw Error('Job receipt does not match the submitted request and reference.');
  if (x.message !== undefined && !text(x.message, 1000))
    throw Error('Invalid job status message.');
  return {
    id: x.id,
    requestId: request.requestId,
    referenceSha256: request.reference.sha256,
    status: x.status as JobReceipt['status'],
    ...(typeof x.message === 'string' ? { message: x.message } : {}),
  };
}
export function validateOffTargetResult(
  value: unknown,
  request: OffTargetRequest,
  expectedJobId?: string,
```





```
): OffTargetResult {
  const x = object(value),
    t = object(x.target);
  if (
    x.schemaVersion !== 1 ||
    x.modelId !== OFFTARGET_MODEL_ID ||
    !text(x.modelVersion) ||
    !text(x.jobId, 128) ||
    !/^[A-Za-z0-9][A-Za-z0-9_-]{0,127}$/.test(x.jobId) ||
    x.requestId !== request.requestId ||
    x.referenceSha256 !== request.reference.sha256 ||
    x.species !== request.reference.species ||
    x.referenceKind !== 'transcript' ||
    x.scope !== 'uploaded-transcript-fasta' ||
    x.scoreType !== 'raw_ranking_score' ||
    (expectedJobId !== undefined && x.jobId !== expectedJobId)
  )
    throw Error(
      'Results must match this job, species, reference checksum and model contract.',
    );
  if (
    t.id !== request.target.id ||
    t.sequence !== request.target.sequence ||
    t.position !== request.target.position ||
    t.motif !== request.target.motif ||
    t.bindingStart !== request.target.bindingStart
  )
    throw Error('Results belong to a different RNA target or binding interval.');
  if (!Array.isArray(x.sites) || x.sites.length > 100000)
    throw Error(
      'Expected at most 100,000 site rows; export larger results from the service.',
    );
  const seen = new Set<string>();
  const sites = x.sites.map((value) => {
    const s = object(value);
    if (
      !text(s.transcript) ||
      !Number.isSafeInteger(s.position) ||
      (s.position as number) < 1 ||
      !['scored', 'unscored', 'insufficient_context'].includes(s.status as string)
    )
      throw Error('Site rows need a transcript ID, 1-based C position and status.');
    if (
      s.context !== null &&
      (typeof s.context !== 'string' || !/^([ACGU]{50})C([ACGU]{50})$/.test(s.context))
    )
      throw Error('A scored RNA context must be 101 nt with central C.');
    if (s.score !== null && (typeof s.score !== 'number' || !Number.isFinite(s.score)))
      throw Error('Raw ranking scores must be finite numbers, or null when unavailable.');
    if (
      (s.status === 'scored' && (s.context === null || s.score === null)) ||
      (s.status !== 'scored' && s.score !== null) ||
      (s.status === 'insufficient_context' && s.context !== null)
    )
      throw Error('Site status is inconsistent with its context and score.');
    if (
      s.bindingStart !== undefined &&
      (!Number.isSafeInteger(s.bindingStart) || (s.bindingStart as number) < 1)
    )
      throw Error('Binding start must be a positive 1-based coordinate.');
    if (
```





```
      s.mismatches !== undefined &&
      (!Number.isSafeInteger(s.mismatches) ||
        (s.mismatches as number) < 0 ||
        (s.mismatches as number) > request.target.motif.length)
    )
      throw Error('Invalid mismatch count.');
    const key = JSON.stringify([s.transcript, s.position, s.bindingStart ?? null]);
    if (seen.has(key)) throw Error('Duplicate site row.');
    seen.add(key);
    return {
      transcript: s.transcript,
      position: s.position as number,
      context: s.context as string | null,
      score: s.score as number | null,
      status: s.status as OffTargetSite['status'],
      ...(s.bindingStart === undefined ? {} : { bindingStart: s.bindingStart as number }),
      ...(s.mismatches === undefined ? {} : { mismatches: s.mismatches as number }),
    };
  });
  const contexts = new Map<string, number>();
  for (const s of sites)
    if (s.context && s.score !== null) {
      const previous = contexts.get(s.context);
      if (previous !== undefined && previous !== s.score)
        throw Error('Identical RNA contexts have conflicting scores.');
      contexts.set(s.context, s.score);
    }
  return {
    schemaVersion: 1,
    jobId: x.jobId,
    requestId: request.requestId,
    modelId: OFFTARGET_MODEL_ID,
    modelVersion: x.modelVersion,
    referenceSha256: request.reference.sha256,
    species: request.reference.species,
    referenceKind: 'transcript',
    scope: 'uploaded-transcript-fasta',
    scoreType: 'raw_ranking_score',
    target: { ...request.target },
    sites,
  };
}
export function resultMatchesInputs(state: OffTargetState, target: WorkflowTarget) {
  const { result, request, reference, job } = state;
  // A restored project has no live FASTA; reselect and verify its sites before reuse.
  return (
    !!state.file &&
    !!result &&
    !!request &&
    !!reference &&
    state.species.trim() === reference.species &&
    request.reference.species === reference.species &&
    result.species === reference.species &&
    targetKey(request.target) === targetKey(target) &&
    targetKey(result.target) === targetKey(target) &&
    request.reference.sha256 === reference.sha256 &&
    result.referenceSha256 === reference.sha256 &&
    result.requestId === request.requestId &&
    result.modelId === request.modelId &&
    result.scoreType === request.scoreType &&
    (!job ||
```





```
      (job.id === result.jobId &&
        job.requestId === request.requestId &&
        job.referenceSha256 === reference.sha256 &&
        job.status === 'succeeded'))
  );
}
export function predictionsFromResult(result: OffTargetResult) {
  const scored = new Map<string, number>();
  for (const s of result.sites)
    if (s.context && s.score !== null) scored.set(s.context, s.score);
  return {
    modelVersion: result.modelVersion,
    referenceSet: result.referenceSha256,
    scoreType: 'raw_ranking_score' as const,
    predictions: [...scored].map(([context, score]) => ({ context, score })),
  };
}
```





lib/scientist-csv.ts

```ts
import type { Candidate, Module, PredictionSet } from './design';

export type ScientistRankRow = {
  candidateId: string;
  protein: string;
  rank: number | null;
  /** Raw ranking-model S_hat; not an ESM-2 score or a probability. */
  score: number | null;
  esm2Score: number | null;
  c295Hat?: number | null;
  c388Hat?: number | null;
  c871Hat?: number | null;
  referenceSha256?: string;
  sourceCheckpointSha256?: string;
  compatibilityVersion?: string;
  source?: string;
  modelVersion: string;
  scope: string;
  status: string;
};

export type ScientistSiteRow = {
  transcript: string;
  /** 1-based coordinate in the supplied reference record. */
  position: number;
  context: string | null;
  score: number | null;
  bindingStart?: number;
  mismatches?: number;
  status: string;
};

export type ScientistResultMetadata = {
  modelVersion?: string;
  referenceHash?: string;
  scope?: string;
  scoreType?: 'ranking_score' | 'raw_ranking_score';
};

export type ScientistPredictionSet = Omit<PredictionSet, 'scoreType'> & {
  scoreType: 'ranking_score' | 'raw_ranking_score';
};

export type ScientistCsvInput = {
  target: { id: string; sequence: string; position: number };
  selected: Candidate;
  plan: {
    sequence: string | null;
    scaffold?: { id: string } | null;
    rows: { repeat: number; base: string; selected: Module | null }[];
  };
  libraryVersion: string;
  reference: {
    name: string;
    species: string;
    kind: 'transcript' | 'genome' | 'unspecified';
```





```
    sha256?: string;
  };
  esmRanks?: ScientistRankRow[];
  activeProteinCandidateId?: string | null;
  predictions?: ScientistPredictionSet | null;
  predictionMetadata?: ScientistResultMetadata;
  externalSites?: ScientistSiteRow[];
  externalMetadata?: ScientistResultMetadata;
};

/**
 * Rectangular CSV schema v1. Coordinates are 1-based in the named sequence record;
 * genome records are not converted to transcript coordinates. Record types:
 * - target: the requested target and selected design (always present).
 * - binding_site: one row for each already computed C site in each binding hit.
 * - binding_hit_no_c: a binding hit without a C site; site fields stay empty.
 * - external_site: every site returned by the external service, kept separately.
 * - protein_candidate: every imported protein ranking row, in input order.
 *
 * ESM-2, raw S_hat and LAMAR scores retain separate columns and original precision.
 * No combined probability is computed. Missing/nonfinite numbers stay empty;
 * genuine zero and negative numeric scores are retained. TRM assignments describe
 * only the selected plan, never an unselected candidate's unprovided modules.
 * When externalSites is supplied, the target score comes only from its reported
 * transcript/position row. Matching contexts at other loci never fill that score.
 * Callers must pass current result snapshots, clearing them when inputs change.
 * Known reference-hash mismatches suppress LAMAR scores, while retaining provenance.
 * Output uses a UTF-8 BOM, RFC 4180 quoting and CRLF record separators. Dangerous
 * string cells receive a leading apostrophe for safe spreadsheet import; numeric
 * values are serialized separately so negative numbers retain their numeric value.
 */
export const SCIENTIST_CSV_COLUMNS = [
  'record_type',
  'target_id',
  'target_position_1based',
  'target_sequence',
  'design_id',
  'protein_candidate_id',
  'protein_selected',
  'protein_sequence',
  'scaffold_id',
  'trm_assignments',
  'library_version',
  'recognition_rna',
  'transcript_id',
  'binding_rna',
  'binding_start_1based',
  'binding_end_1based',
  'site_position_1based',
  'site_offset_nt',
  'binding_mismatches',
  'context_101nt',
  'esm_rank',
  's_hat_raw',
  'esm2_score',
  'c295_hat',
  'c388_hat',
  'c871_hat',
  'esm_model_version',
  'esm_scope',
  'esm_status',
```





```
  'esm_reference_sha256',
  'esm_source',
  'esm_checkpoint_sha256',
  'esm_compatibility_version',
  'lamar_score',
  'lamar_score_available',
  'lamar_score_type',
  'lamar_status',
  'lamar_model_version',
  'lamar_reference_set',
  'lamar_scope',
  'lamar_reference_sha256',
  'reference_name',
  'reference_species',
  'reference_kind',
  'reference_sha256',
  'coordinate_system',
] as const;

type Cell = string | number | boolean | null | undefined;
type CsvRow = Partial<Record<(typeof SCIENTIST_CSV_COLUMNS)[number], Cell>>;

function finite(value: number | null | undefined): number | null {
  return typeof value === 'number' && Number.isFinite(value) ? value : null;
}

function cell(value: Cell): string {
  if (value === null || value === undefined) return '';
  if (typeof value === 'number') return Number.isFinite(value) ? String(value) : '';
  let result = String(value);
  // Whitespace/control prefixes can hide spreadsheet formula triggers.
  if (typeof value === 'string' && /^[\s\u0000-\u001f]*[=+\-@]/u.test(result))
    result = `'${result}`;
  return /[",\r\n]/.test(result) ? `"${result.replaceAll('"', '""')}"` : result;
}

function mismatchedReference(
  input: ScientistCsvInput,
  metadata?: ScientistResultMetadata,
): boolean {
  return !!(
    input.reference.sha256 &&
    metadata?.referenceHash &&
    input.reference.sha256.toLowerCase() !== metadata.referenceHash.toLowerCase()
  );
}

function rankColumns(rank?: ScientistRankRow): CsvRow {
  return {
    esm_rank: finite(rank?.rank),
    s_hat_raw: finite(rank?.score),
    esm2_score: finite(rank?.esm2Score),
    esm_model_version: rank?.modelVersion,
    esm_scope: rank?.scope,
    esm_status: rank?.status,
    c295_hat: finite(rank?.c295Hat),
    c388_hat: finite(rank?.c388Hat),
    c871_hat: finite(rank?.c871Hat),
    esm_reference_sha256: rank?.referenceSha256,
    esm_source: rank?.source,
    esm_checkpoint_sha256: rank?.sourceCheckpointSha256,
```





```
    esm_compatibility_version: rank?.compatibilityVersion,
  };
}

/** Export already-computed results. Does not run or validate biological models. */
export function buildScientistCsv(input: ScientistCsvInput): string {
  const { target, selected, plan, reference } = input;
  const ranks = input.esmRanks ?? [];
  const active = ranks.find(
    (row) =>
      row.candidateId === input.activeProteinCandidateId &&
      !!plan.sequence &&
      row.protein === plan.sequence,
  );
  const trms = plan.rows
    .map((row) => {
      const selectedModule = row.selected;
      const source = selectedModule?.sourceRepeat ?? selectedModule?.nativeRepeat;
      return `P${
        row.repeat
      }:${
        row.base
      }:${
        selectedModule?.trm ?? ''
      }:module=${
        selectedModule?.id ?? ''
      }${
        source === undefined ? '' : `:source=R${source}`
      }`;
    })
    .join('; ');
  const common: CsvRow = {
    target_id: target.id,
    target_position_1based: target.position,
    target_sequence: target.sequence,
    design_id: selected.id,
    library_version: input.libraryVersion,
    recognition_rna: selected.motif,
    reference_name: reference.name,
    reference_species: reference.species,
    reference_kind: reference.kind,
    reference_sha256: reference.sha256,
    coordinate_system: '1-based sequence-record coordinates',
  };
  const chosen: CsvRow = {
    ...common,
    protein_candidate_id: active?.candidateId,
    protein_selected: !!plan.sequence,
    protein_sequence: plan.sequence,
    scaffold_id: plan.scaffold?.id,
    trm_assignments: trms,
    ...rankColumns(active),
  };
  const scores = new Map(
    input.predictions?.predictions.map((row) => [row.context, row.score]) ?? [],
  );
  const predictionMismatch = mismatchedReference(input, input.predictionMetadata);
  const contextColumns = (context: string | null): CsvRow => {
    const raw = context === null ? null : finite(scores.get(context));
    const score = predictionMismatch ? null : raw;
    return {
```





```
      context_101nt: context,
      lamar_score: score,
      lamar_score_available: score !== null,
      lamar_score_type: input.predictions?.scoreType,
      lamar_status: predictionMismatch
        ? 'reference_mismatch'
        : score !== null
          ? 'available'
          : context === null
            ? 'context_unavailable'
            : 'score_unavailable',
      lamar_model_version:
        input.predictions?.modelVersion ?? input.predictionMetadata?.modelVersion,
      lamar_reference_set: input.predictions?.referenceSet,
      lamar_scope: input.predictionMetadata?.scope,
      lamar_reference_sha256: input.predictionMetadata?.referenceHash,
    };
  };
  const fullTargetContext =
    Number.isInteger(target.position) &&
    target.position > 50 &&
    target.position + 50 <= target.sequence.length
      ? target.sequence.slice(target.position - 51, target.position + 50)
      : null;
  const externalMismatch = mismatchedReference(input, input.externalMetadata);
  const externalScoreColumns = (site?: ScientistSiteRow): CsvRow => {
    const score = externalMismatch ? null : finite(site?.score);
    return {
      context_101nt: site?.context,
      lamar_score: score,
      lamar_score_available: score !== null,
      lamar_score_type: input.externalMetadata?.scoreType,
      lamar_status: externalMismatch
        ? 'reference_mismatch'
        : (site?.status ?? 'site_unreported'),
      lamar_model_version: input.externalMetadata?.modelVersion,
      lamar_scope: input.externalMetadata?.scope,
      lamar_reference_sha256: input.externalMetadata?.referenceHash,
    };
  };
  const targetScores =
    input.externalSites === undefined
      ? contextColumns(fullTargetContext)
      : externalScoreColumns(
          input.externalSites.find(
            (site) => site.transcript === target.id && site.position === target.position,
          ),
        );
  const rows: CsvRow[] = [
    {
      ...chosen,
      record_type: 'target',
      transcript_id: target.id,
      binding_rna: selected.motif,
      binding_start_1based: selected.start,
      binding_end_1based: selected.end,
      site_position_1based: target.position,
      site_offset_nt: selected.offset,
      ...targetScores,
    },
  ];
```





```
  for (const hit of selected.hits) {
    const hitRow: CsvRow = {
      ...chosen,
      transcript_id: hit.transcript,
      binding_start_1based: hit.start,
      binding_rna: hit.sequence,
      binding_end_1based: hit.start + hit.sequence.length - 1,
      binding_mismatches: hit.mismatches,
    };
    if (!hit.editingSites.length) {
      rows.push({
        ...hitRow,
        record_type: 'binding_hit_no_c',
        ...contextColumns(null),
        lamar_status: 'no_c_site',
      });
    } else {
      for (const site of hit.editingSites)
        rows.push({
          ...hitRow,
          record_type: 'binding_site',
          site_position_1based: site.position,
          site_offset_nt: site.offset,
          ...contextColumns(site.context),
        });
    }
  }
  for (const site of input.externalSites ?? []) {
    rows.push({
      ...chosen,
      record_type: 'external_site',
      transcript_id: site.transcript,
      binding_start_1based: site.bindingStart,
      binding_mismatches: site.mismatches,
      site_position_1based: site.position,
      ...externalScoreColumns(site),
    });
  }
  for (const rank of ranks) {
    const isSelected = rank === active;
    rows.push({
      ...common,
      record_type: 'protein_candidate',
      protein_candidate_id: rank.candidateId,
      protein_selected: isSelected,
      protein_sequence: rank.protein,
      scaffold_id: isSelected ? plan.scaffold?.id : null,
      trm_assignments: isSelected ? trms : null,
      ...rankColumns(rank),
    });
  }
  return (
    '\uFEFF' +
    [
      SCIENTIST_CSV_COLUMNS.join(','),
      ...rows.map((row) =>
        SCIENTIST_CSV_COLUMNS.map((column) => cell(row[column])).join(','),
      ),
    ].join('\r\n') +
    '\r\n'
  );
}
```





lib/workflow-service.ts

```ts
/** Same-origin transport for an administrator-configured model service. */
export interface WorkflowServiceEnv {
  WORKFLOW_MODEL_URL?: string;
  WORKFLOW_MODEL_TOKEN?: string;
}

export interface WorkflowHealth {
  available: boolean;
  reason?: string;
  models: { esm2: boolean; lamar: boolean };
}

interface GatewayOptions {
  fetch?: typeof fetch;
  healthTimeoutMs?: number;
  requestTimeoutMs?: number;
  jsonLimitBytes?: number;
  multipartLimitBytes?: number;
}

const PREFIX = '/api/workflow';
const JSON_LIMIT = 2 * 1024 * 1024;
// The upstream service must cap the FASTA part at 100 MiB; this envelope also
// allows 64 KiB for metadata, field headers, and multipart boundaries.
const MULTIPART_LIMIT = 100 * 1024 * 1024 + 64 * 1024;
const HEALTH_LIMIT = 64 * 1024;

class BodyTooLarge extends Error {}
class RequestTimedOut extends Error {}

function jsonResponse(body: unknown, status = 200): Response {
  return Response.json(body, {
    status,
    headers: { 'Cache-Control': 'no-store', 'X-Content-Type-Options': 'nosniff' },
  });
}

function unavailable(reason: string): WorkflowHealth {
  return { available: false, reason, models: { esm2: false, lamar: false } };
}

function failure(status: number, message: string, health = false): Response {
  return jsonResponse(health ? unavailable(message) : { error: message }, status);
}

function serviceBase(value: string): URL | null {
  try {
    const base = new URL(value);
    if (
      base.protocol !== 'https:' ||
      base.username ||
      base.password ||
      base.search ||
      base.hash
    )
      return null;
```

```
    base.pathname = base.pathname.replace(/\/+$/, '') + '/';
    return base;
  } catch {
    return null;
  }
}

async function readBounded(
  body: ReadableStream<Uint8Array> | null,
  limit: number,
): Promise<Uint8Array> {
  if (!body) return new Uint8Array();
  const reader = body.getReader();
  const chunks: Uint8Array[] = [];
  let size = 0;
  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      size += value.byteLength;
      if (size > limit) {
        void reader.cancel().catch(() => {});
        throw new BodyTooLarge();
      }
      chunks.push(value);
    }
  } finally {
    reader.releaseLock();
  }
  const bytes = new Uint8Array(size);
  let offset = 0;
  for (const chunk of chunks) {
    bytes.set(chunk, offset);
    offset += chunk.byteLength;
  }
  return bytes;
}

function cappedStream(
  body: ReadableStream<Uint8Array>,
  limit: number,
  overflow: () => void,
): { stream: ReadableStream<Uint8Array>; completed: Promise<void> } {
  const reader = body.getReader();
  let size = 0;
  let finish!: () => void;
  let fail!: (reason: unknown) => void;
  const completed = new Promise<void>((resolve, reject) => {
    finish = resolve;
    fail = reject;
  });
  // A stream can fail while fetch is still waiting for upstream headers.
  void completed.catch(() => {});
  const stream = new ReadableStream<Uint8Array>({
    async pull(controller) {
      try {
        const { done, value } = await reader.read();
        if (done) {
          controller.close();
          reader.releaseLock();
          finish();
```





```
          return;
        }
        size += value.byteLength;
        if (size > limit) {
          overflow();
          void reader.cancel().catch(() => {});
          const error = new BodyTooLarge();
          fail(error);
          controller.error(error);
          return;
        }
        controller.enqueue(value);
      } catch (error) {
        fail(error);
        controller.error(error);
      }
    },
    cancel(reason) {
      fail(reason ?? new Error('Upload interrupted'));
      return reader.cancel(reason);
    },
  });
  return { stream, completed };
}

function normalizeHealth(value: unknown): WorkflowHealth | null {
  if (!value || typeof value !== 'object' || Array.isArray(value)) return null;
  const health = value as Record<string, unknown>;
  if (
    typeof health.available !== 'boolean' ||
    !health.models ||
    typeof health.models !== 'object' ||
    Array.isArray(health.models)
  )
    return null;
  const models = health.models as Record<string, unknown>;
  if (typeof models.esm2 !== 'boolean' || typeof models.lamar !== 'boolean') return null;
  if (
    health.reason !== undefined &&
    (typeof health.reason !== 'string' || health.reason.length > 500)
  )
    return null;
  return {
    available: health.available,
    ...(typeof health.reason === 'string' ? { reason: health.reason } : {}),
    models: { esm2: models.esm2, lamar: models.lamar },
  };
}

/** Returns null for unrelated paths so the existing application router retains ownership. */
export async function handleWorkflowRequest(
  request: Request,
  env: WorkflowServiceEnv,
  options: GatewayOptions = {},
): Promise<Response | null> {
  const url = new URL(request.url);
  if (url.pathname !== PREFIX && !url.pathname.startsWith(PREFIX + '/')) return null;
  const path = url.pathname.slice(PREFIX.length);
  const health = path === '/health';
  const ranking = path === '/rank';
  const createJob = path === '/jobs';
```





```
  const readJob = /^\/jobs\/[A-Za-z0-9][A-Za-z0-9_-]{0,127}(?:\/result)?$/.test(path);
  if (!health && !ranking && !createJob && !readJob)
    return failure(404, 'Unknown workflow endpoint.');
  if (url.search)
    return failure(400, 'Workflow query parameters are not supported.', health);
  const method = ranking || createJob ? 'POST' : 'GET';
  if (request.method !== method) {
    const response = failure(405, 'Method not allowed.', health);
    response.headers.set('Allow', method);
    return response;
  }
  if (method === 'POST' && request.headers.get('Origin') !== url.origin) {
    return failure(403, 'A same-origin request is required.');
  }
  if (!env.WORKFLOW_MODEL_URL) {
    return failure(health ? 200 : 503, 'Model service is not configured.', health);
  }
  const base = serviceBase(env.WORKFLOW_MODEL_URL);
  if (!base)
    return failure(health ? 200 : 503, 'Model service configuration is invalid.', health);

  const controller = new AbortController();
  const deadline = health
    ? (options.healthTimeoutMs ?? 15_000)
    : (options.requestTimeoutMs ?? 30_000);
  let timedOut = false;
  let overflowed = false;
  let timeout: ReturnType<typeof setTimeout> | undefined;
  const onAbort = () => controller.abort();
  request.signal.addEventListener('abort', onAbort, { once: true });
  if (request.signal.aborted) controller.abort();

  const operation = async (): Promise<Response> => {
    let uploadCompleted: Promise<void> | undefined;
    const headers = new Headers({ Accept: 'application/json' });
    if (env.WORKFLOW_MODEL_TOKEN)
      headers.set('Authorization', `Bearer ${env.WORKFLOW_MODEL_TOKEN}`);
    const init: RequestInit & { duplex?: 'half' } = {
      method,
      headers,
      redirect: 'error',
      signal: controller.signal,
    };
    if (method === 'POST') {
      const contentType = request.headers.get('Content-Type') ?? '';
      const validType = ranking
        ? /^application\/json(?:\s*;|$)/i.test(contentType)
        : /^multipart\/form-data\s*;/i.test(contentType) &&
          /(?:^|;)\s*boundary=(?:"[^"\r\n]{1,70}"|[^;\s]{1,70})(?:\s*;|\s*$)/i.test(
            contentType,
          );
      if (!validType)
        return failure(
          415,
          ranking
            ? 'A JSON request is required.'
            : 'A multipart FASTA upload is required.',
        );
      const limit = ranking
        ? (options.jsonLimitBytes ?? JSON_LIMIT)
        : (options.multipartLimitBytes ?? MULTIPART_LIMIT);
```





```
      const contentLength = request.headers.get('Content-Length');
      if (contentLength !== null && !/^\d+$/.test(contentLength))
        return failure(400, 'Invalid request length.');
      if (contentLength !== null && Number(contentLength) > limit)
        return failure(413, 'Request body exceeds the upload limit.');
      if (!request.body) return failure(400, 'A request body is required.');
      if (ranking) {
        const bytes = await readBounded(request.body, limit);
        if (controller.signal.aborted) throw new Error('Request aborted');
        let payload: unknown;
        let json: string;
        try {
          json = new TextDecoder('utf-8', { fatal: true }).decode(bytes);
          payload = JSON.parse(json);
        } catch {
          return failure(400, 'Invalid JSON request.');
        }
        if (!payload || typeof payload !== 'object' || Array.isArray(payload))
          return failure(400, 'A JSON object is required.');
        headers.set('Content-Type', 'application/json');
        init.body = json;
      } else {
        headers.set('Content-Type', contentType);
        const upload = cappedStream(request.body, limit, () => {
          overflowed = true;
          controller.abort();
        });
        init.body = upload.stream;
        uploadCompleted = upload.completed;
        init.duplex = 'half';
      }
    }
    const upstream = await (options.fetch ?? fetch)(new URL(path.slice(1), base), init);
    if (overflowed) throw new BodyTooLarge();
    if (!upstream.ok) {
      void upstream.body?.cancel().catch(() => {});
      controller.abort();
      return failure(
        upstream.status,
        health ? 'Model service health check failed.' : 'Model service request failed.',
        health,
      );
    }
    // An upstream early acknowledgement cannot bypass the upload byte cap.
    if (uploadCompleted) await uploadCompleted;
    if (
      !/^application\/json(?:\s*;|$)/i.test(upstream.headers.get('Content-Type') ?? '')
    ) {
      void upstream.body?.cancel().catch(() => {});
      return failure(502, 'Model service returned an invalid response.', health);
    }
    if (health) {
      const bytes = await readBounded(upstream.body, HEALTH_LIMIT);
      let data: unknown;
      try {
        data = JSON.parse(new TextDecoder('utf-8', { fatal: true }).decode(bytes));
      } catch {
        return failure(502, 'Model service returned invalid health data.', true);
      }
      const normalized = normalizeHealth(data);
      return normalized
```





```
        ? jsonResponse(normalized, upstream.status)
        : failure(502, 'Model service returned invalid health data.', true);
    }
    return new Response(upstream.body, {
      status: upstream.status,
      headers: {
        'Content-Type': 'application/json',
        'Cache-Control': 'no-store',
        'X-Content-Type-Options': 'nosniff',
      },
    });
  };

  try {
    return await Promise.race([
      operation(),
      new Promise<Response>((_, reject) => {
        timeout = setTimeout(() => {
          timedOut = true;
          controller.abort();
          reject(new RequestTimedOut());
        }, deadline);
      }),
    ]);
  } catch (error) {
    if (overflowed || (error instanceof BodyTooLarge && !health))
      return failure(413, 'Request body exceeds the upload limit.');
    if (timedOut || error instanceof RequestTimedOut)
      return failure(504, 'Model service request timed out.', health);
    return failure(502, 'Model service could not be reached.', health);
  } finally {
    if (timeout !== undefined) clearTimeout(timeout);
    request.signal.removeEventListener('abort', onAbort);
  }
}
```





lib/local-downloads.ts

```ts
import bundledRelease from '../data/local-download-release.json' with { type: 'json' };

type AssetFetcher = { fetch(request: Request): Promise<Response> };
export type DownloadRelease = {
  version: string;
  fileName: string;
  bytes: number;
  sha256: string;
  urls: { windows: string; macos: string };
  parts: Array<{ path: string; bytes: number; sha256: string }>;
};

const release = bundledRelease as DownloadRelease;

function matchesEtag(value: string | null, etag: string) {
  return (
    value
      ?.split(',')
      .some((item) => item.trim() === '*' || item.trim().replace(/^W\//, '') === etag) ??
    false
  );
}

function byteRange(value: string, size: number): { start: number; end: number } | null {
  const match = /^bytes=(\d*)-(\d*)$/.exec(value);
  if (!match || (!match[1] && !match[2])) return null;
  if (!match[1]) {
    const suffix = Number(match[2]);
    if (!Number.isSafeInteger(suffix) || suffix <= 0) return null;
    return { start: Math.max(0, size - suffix), end: size - 1 };
  }
  const start = Number(match[1]),
    end = match[2] ? Number(match[2]) : size - 1;
  if (
    !Number.isSafeInteger(start) ||
    !Number.isSafeInteger(end) ||
    start < 0 ||
    start >= size ||
    end < start
  )
    return null;
  return { start, end: Math.min(end, size - 1) };
}

/** Stream the immutable package through ASSETS with bounded memory and native
 * download/range semantics. Pieces stay below the host's individual asset cap.
 * Does not access model inputs, run inference, or accept uploads. */
export async function handleLocalDownload(
  request: Request,
  assets: AssetFetcher,
  current: DownloadRelease = release,
): Promise<Response | null> {
  const url = new URL(request.url);
  if (url.pathname !== current.urls.windows && url.pathname !== current.urls.macos)
    return null;
  if (!assets || typeof assets.fetch !== 'function')
```





```
    return new Response('Download assets are temporarily unavailable.', {
      status: 503,
      headers: { 'Cache-Control': 'no-store' },
    });
  if (
    !Number.isSafeInteger(current.bytes) ||
    current.bytes <= 0 ||
    !current.parts.length ||
    current.parts.some((part) => !Number.isSafeInteger(part.bytes) || part.bytes <= 0) ||
    current.parts.reduce((sum, part) => sum + part.bytes, 0) !== current.bytes
  )
    return new Response('Download manifest is inconsistent.', {
      status: 503,
      headers: { 'Cache-Control': 'no-store' },
    });
  const etag = `"${current.sha256}"`;
  const headers = new Headers({
    'Content-Type': 'application/zip',
    'Content-Disposition': `attachment; filename="${current.fileName}"`,
    'Accept-Ranges': 'bytes',
    'ETag': etag,
    'X-Content-Type-Options': 'nosniff',
    'Cache-Control': 'public, max-age=31536000, immutable',
  });
  if (request.method !== 'GET' && request.method !== 'HEAD')
    return new Response(null, { status: 405, headers: { Allow: 'GET, HEAD' } });
  if (url.search)
    return new Response('Download URLs do not accept parameters.', {
      status: 400,
      headers: { 'Cache-Control': 'no-store' },
    });
  if (matchesEtag(request.headers.get('If-None-Match'), etag))
    return new Response(null, { status: 304, headers });
  let selected = { start: 0, end: current.bytes - 1 },
    status = 200;
  const requested = request.headers.get('Range');
  const ifRange = request.headers.get('If-Range');
  if (request.method === 'GET' && requested && (!ifRange || ifRange === etag)) {
    const parsed = byteRange(requested, current.bytes);
    if (!parsed)
      return new Response(null, {
        status: 416,
        headers: {
          ...Object.fromEntries(headers),
          'Content-Range': `bytes */${current.bytes}`,
          'Content-Length': '0',
        },
      });
    selected = parsed;
    status = 206;
    headers.set(
      'Content-Range',
      `bytes ${selected.start}-${selected.end}/${current.bytes}`,
    );
  }
  headers.set('Content-Length', String(selected.end - selected.start + 1));
  if (request.method === 'HEAD') return new Response(null, { status, headers });
  const segments: Array<{ path: string; start: number; end: number; size: number }> = [];
  let offset = 0;
  for (const part of current.parts) {
    const end = offset + part.bytes - 1;
```





```
    if (selected.start <= end && selected.end >= offset)
      segments.push({
        path: part.path,
        start: Math.max(0, selected.start - offset),
        end: Math.min(part.bytes - 1, selected.end - offset),
        size: part.bytes,
      });
    offset += part.bytes;
  }
  if (offset !== current.bytes || !segments.length)
    return new Response('Download manifest is inconsistent.', {
      status: 503,
      headers: { 'Cache-Control': 'no-store' },
    });
  let next = 0,
    reader: ReadableStreamDefaultReader<Uint8Array> | null = null,
    skip = 0,
    remaining = 0,
    cancelled = false;
  const body = new ReadableStream<Uint8Array>({
    async pull(controller) {
      try {
        while (!cancelled) {
          if (!reader) {
            if (next >= segments.length) {
              controller.close();
              return;
            }
            const part = segments[next++];
            const partResponse = await assets.fetch(
              new Request(new URL(part.path, url), {
                method: 'GET',
                headers: {
                  Range: `bytes=${part.start}-${part.end}`,
                  'Accept-Encoding': 'identity',
                },
              }),
            );
            if (cancelled) {
              await partResponse.body?.cancel();
              return;
            }
            if (
              (partResponse.status !== 200 && partResponse.status !== 206) ||
              !partResponse.body
            ) {
              await partResponse.body?.cancel();
              throw Error('A package asset is unavailable. Retry the download.');
            }
            const expectedLength =
              partResponse.status === 206 ? part.end - part.start + 1 : part.size;
            const assetLength = partResponse.headers.get('Content-Length');
            if (
              (assetLength !== null && Number(assetLength) !== expectedLength) ||
              /^text\/html\b/i.test(partResponse.headers.get('Content-Type') ?? '') ||
              !['', 'identity'].includes(
                partResponse.headers.get('Content-Encoding') ?? '',
              )
            ) {
              await partResponse.body.cancel();
              throw Error('A package asset returned invalid download metadata.');
```





```
            }
            if (
              partResponse.status === 206 &&
              partResponse.headers.get('Content-Range') !==
                `bytes ${part.start}-${part.end}/${part.size}`
            ) {
              await partResponse.body.cancel();
              throw Error('A package asset returned a mismatched range.');
            }
            reader = partResponse.body.getReader();
            skip = partResponse.status === 206 ? 0 : part.start;
            remaining = part.end - part.start + 1;
          }
          const { value, done } = await reader.read();
          if (cancelled) return;
          if (done) {
            reader.releaseLock();
            reader = null;
            throw Error('The package download was truncated. Retry the download.');
          }
          if (!value?.byteLength) continue;
          const begin = Math.min(skip, value.byteLength);
          skip -= begin;
          const length = Math.min(remaining, value.byteLength - begin);
          if (!length) continue;
          remaining -= length;
          controller.enqueue(value.subarray(begin, begin + length));
          if (remaining === 0) {
            const finished = reader;
            reader = null;
            try {
              await finished.cancel();
            } finally {
              finished.releaseLock();
            }
            if (!cancelled && next >= segments.length) controller.close();
          }
          return;
        }
      } catch (error) {
        console.error(
          'ORCA package download failed:',
          error instanceof Error ? error.message : 'Stream failure.',
        );
        if (reader) {
          await reader.cancel().catch(() => {});
          reader.releaseLock();
          reader = null;
        }
        if (!cancelled) controller.error(error);
      }
    },
    async cancel() {
      cancelled = true;
      const active = reader;
      reader = null;
```





```
      if (active) {
        await active.cancel().catch(() => {});
        active.releaseLock();
      }
    },
  });
  return new Response(body, { status, headers });
}
```





scripts/run-bounded.mjs

```
import { spawn } from 'node:child_process';

function milliseconds(value) {
  const match = /^(\d+(?:\.\d+)?)([smh]?)$/.exec(value);
  if (!match || Number(match[1]) <= 0)
    throw new Error(`Invalid positive timeout: ${value}`);
  return Number(match[1]) * { s: 1000, m: 60000, h: 3600000 }[match[2] || 's'];
}

const [command, ...args] = process.argv.slice(2);
if (!command) throw new Error('A command is required.');
const deadline = milliseconds(process.env.SITES_BUILD_TIMEOUT || '3m');
const grace = milliseconds(process.env.SITES_BUILD_KILL_AFTER || '10s');
const grouped = process.platform !== 'win32';
const child = spawn(command, args, { stdio: 'inherit', detached: grouped });
let timedOut = false;
let forceTimer;
const stop = (signal) => {
  if (!child.pid) return;
  try {
    if (grouped) process.kill(-child.pid, signal);
    else child.kill(signal);
  } catch (error) {
    if (error.code !== 'ESRCH') throw error;
  }
};
const onTerm = () => stop('SIGTERM');
const onInt = () => stop('SIGINT');
process.on('SIGTERM', onTerm);
process.on('SIGINT', onInt);
const timer = setTimeout(() => {
  timedOut = true;
  console.error('Build time limit reached; stopping the build.');
  stop('SIGTERM');
  forceTimer = setTimeout(() => stop('SIGKILL'), grace);
}, deadline);
child.on('error', (error) => console.error(`Cannot start build: ${error.message}`));
child.on('close', (code, signal) => {
  clearTimeout(timer);
  clearTimeout(forceTimer);
  process.removeListener('SIGTERM', onTerm);
  process.removeListener('SIGINT', onInt);
  process.exitCode = timedOut
    ? 124
    : signal === 'SIGINT'
      ? 130
      : signal
        ? 143
        : (code ?? 1);
});
```





tests/scientist-csv.test.mjs

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { buildScientistCsv, SCIENTIST_CSV_COLUMNS } from '../lib/scientist-csv.ts';

const context = 'A'.repeat(50) + 'C' + 'G'.repeat(50);
function fixture() {
  return {
    target: { id: 'target', sequence: context, position: 51 },
    selected: {
      id: 'R8-D2',
      n: 8,
      offset: 2,
      start: 42,
      end: 49,
      motif: 'AAAAAAAA',
      hits: [],
      exact: 0,
      closest: 3,
      pareto: true,
    },
    plan: {
      sequence: 'ACDE',
      scaffold: { id: 'scaffold-v1' },
      rows: [
        {
          repeat: 1,
          base: 'A',
          selected: {
            id: 'module-1',
            repeat: 1,
            base: 'A',
            trm: 'XXX',
            sequence: 'ACDE',
            evidence: 'fixture',
            sourceRepeat: 8,
          },
        },
        { repeat: 2, base: 'G', selected: null },
      ],
    },
    libraryVersion: 'library-v1',
    reference: {
      name: 'fixture-reference',
      species: 'fixture-species',
      kind: 'transcript',
      sha256: 'a'.repeat(64),
    },
  };
}

// Independent RFC 4180 reader so tests verify cell boundaries and embedded lines.
function parse(csv) {
  assert.equal(csv[0], '\uFEFF');
  const source = csv.slice(1),
    rows = [];
  let row = [],
```





```
    value = '',
    quoted = false;
  for (let i = 0; i < source.length; i++) {
    const char = source[i];
    if (char === '"') {
      if (quoted && source[i + 1] === '"') {
        value += '"';
        i++;
      } else quoted = !quoted;
    } else if (char === ',' && !quoted) {
      row.push(value);
      value = '';
    } else if (char === '\r' && source[i + 1] === '\n' && !quoted) {
      row.push(value);
      rows.push(row);
      row = [];
      value = '';
      i++;
    } else value += char;
  }
  assert.equal(quoted, false);
  assert.equal(value, '');
  assert.equal(row.length, 0);
  const [header, ...data] = rows;
  assert.deepEqual(header, [...SCIENTIST_CSV_COLUMNS]);
  for (const cells of data) assert.equal(cells.length, header.length);
  return data.map((cells) =>
    Object.fromEntries(header.map((key, index) => [key, cells[index]])),
  );
}

test('empty hits still export the requested target, complete provenance and selected plan', () =>
{
  const input = fixture(),
    before = structuredClone(input);
  const csv = buildScientistCsv(input),
    rows = parse(csv);
  assert.equal(rows.length, 1);
  assert.equal(rows[0].record_type, 'target');
  assert.equal(rows[0].target_position_1based, '51');
  assert.equal(rows[0].site_position_1based, '51');
  assert.equal(rows[0].binding_start_1based, '42');
  assert.equal(rows[0].binding_end_1based, '49');
  assert.equal(rows[0].protein_sequence, 'ACDE');
  assert.equal(rows[0].scaffold_id, 'scaffold-v1');
  assert.equal(
    rows[0].trm_assignments,
    'P1:A:XXX:module=module-1:source=R8; P2:G::module=',
  );
  assert.equal(rows[0].reference_sha256, input.reference.sha256);
  assert.equal(rows[0].reference_species, 'fixture-species');
  assert.equal(rows[0].library_version, 'library-v1');
  assert.equal(rows[0].coordinate_system, '1-based sequence-record coordinates');
  assert.equal(rows[0].lamar_score, '');
  assert.equal(rows[0].lamar_score_available, 'false');
  assert.deepEqual(input, before);
  assert.ok(csv.endsWith('\r\n'));
  assert.ok(!csv.replaceAll('\r\n', '').includes('\n'));
});

test('zero LAMAR scores survive while absent scores and missing contexts stay blank', () => {
```





```
  const input = fixture();
  input.predictions = {
    modelVersion: 'LAMAR-v1',
    referenceSet: 'fixed-population',
    scoreType: 'ranking_score',
    predictions: [{ context, score: 0 }],
  };
  input.predictionMetadata = {
    referenceHash: input.reference.sha256,
    scope: 'reference subset',
  };
  input.selected.hits = [
    {
      transcript: 'multi-C',
      start: 8,
      sequence: 'AAAAAAAA',
      mismatches: 0,
      editingSites: [
        { position: 17, offset: 2, context },
        { position: 18, offset: 3, context: 'G'.repeat(50) + 'C' + 'A'.repeat(50) },
        { position: 19, offset: 4, context: null },
      ],
    },
  ];
  const rows = parse(buildScientistCsv(input));
  assert.equal(rows.length, 4);
  assert.deepEqual(
    rows.slice(1).map((row) => row.site_position_1based),
    ['17', '18', '19'],
  );
  assert.deepEqual(
    rows.slice(1).map((row) => row.binding_end_1based),
    ['15', '15', '15'],
  );
  assert.deepEqual(
    rows.map((row) => row.lamar_score),
    ['0', '0', '', ''],
  );
  assert.deepEqual(
    rows.map((row) => row.lamar_score_available),
    ['true', 'true', 'false', 'false'],
  );
  assert.equal(rows[2].lamar_status, 'score_unavailable');
  assert.equal(rows[3].lamar_status, 'context_unavailable');
  assert.equal(rows[0].lamar_model_version, 'LAMAR-v1');
  assert.equal(rows[0].lamar_reference_set, 'fixed-population');
  assert.equal(rows[0].lamar_scope, 'reference subset');
});

test('hits with no C sites retain their binding interval and have no invented site', () => {
  const input = fixture();
  input.selected.hits = [
    {
      transcript: 'no-C',
      start: 2,
      sequence: 'AAAAAAAG',
      mismatches: 1,
      editingSites: [],
    },
  ];
  const row = parse(buildScientistCsv(input))[1];
```





```
  assert.equal(row.record_type, 'binding_hit_no_c');
  assert.equal(row.transcript_id, 'no-C');
  assert.equal(row.binding_start_1based, '2');
  assert.equal(row.binding_end_1based, '9');
  assert.equal(row.binding_mismatches, '1');
  assert.equal(row.binding_rna, 'AAAAAAAG');
  assert.equal(row.recognition_rna, 'AAAAAAAA');
  assert.equal(row.site_position_1based, '');
  assert.equal(row.site_offset_nt, '');
  assert.equal(row.lamar_status, 'no_c_site');
});

test('S_hat, ESM-2 and LAMAR are distinct; protein candidates preserve selection and scope', () =>
 {
  const input = fixture();
  input.esmRanks = [
    {
      candidateId: 'chosen',
      protein: 'ACDE',
      rank: 1,
      score: -0.25,
      esm2Score: -1.75,
      modelVersion: 'rank-v2',
      scope: 'provided candidates',
      status: 'scored',
    },
    {
      candidateId: 'other',
      protein: 'FGHI',
      rank: 2,
      score: 0,
      esm2Score: null,
      modelVersion: 'rank-v2',
      scope: 'provided candidates',
      status: 'partial',
    },
  ];
  input.activeProteinCandidateId = 'chosen';
  input.predictions = {
    modelVersion: 'LAMAR-v1',
    referenceSet: 'fixed-population',
    scoreType: 'ranking_score',
    predictions: [{ context, score: 0.6 }],
  };
  const [target, selected, other] = parse(buildScientistCsv(input));
  assert.equal(target.s_hat_raw, '-0.25');
  assert.equal(target.esm2_score, '-1.75');
  assert.equal(target.lamar_score, '0.6');
  assert.equal(selected.record_type, 'protein_candidate');
  assert.equal(selected.protein_selected, 'true');
  assert.equal(selected.esm_rank, '1');
  assert.equal(selected.esm_model_version, 'rank-v2');
  assert.equal(selected.esm_scope, 'provided candidates');
  assert.equal(selected.trm_assignments, target.trm_assignments);
  assert.equal(selected.lamar_score, '');
  assert.equal(other.protein_selected, 'false');
  assert.equal(other.protein_sequence, 'FGHI');
  assert.equal(other.s_hat_raw, '0');
  assert.equal(other.esm2_score, '');
  assert.equal(other.trm_assignments, '');
  assert.equal(other.scaffold_id, '');
```





```
});

test('stale protein matches do not label a different protein selected or attach its score', () =>
{
  const input = fixture();
  input.activeProteinCandidateId = 'old';
  input.esmRanks = [
    {
      candidateId: 'old',
      protein: 'FGHI',
      rank: 1,
      score: 0.8,
      esm2Score: -1,
      modelVersion: 'rank-v1',
      scope: 'old',
      status: 'scored',
    },
  ];
  const [target, candidate] = parse(buildScientistCsv(input));
  assert.equal(target.protein_candidate_id, '');
  assert.equal(target.s_hat_raw, '');
  assert.equal(candidate.protein_selected, 'false');
  assert.equal(candidate.trm_assignments, '');
});

test('all assay outputs and ranking provenance retain independent unrounded values', () => {
  const input = fixture();
  input.activeProteinCandidateId = 'chosen';
  input.esmRanks = [
    {
      candidateId: 'chosen',
      protein: 'ACDE',
      rank: 1,
      score: -0.01234567890123456,
      esm2Score: -2.987654321098765,
      c295Hat: 0,
      c388Hat: -0.1234567890123456,
      c871Hat: 1.234567890123456,
      referenceSha256: 'd'.repeat(64),
      source: 'imported',
      sourceCheckpointSha256: 'e'.repeat(64),
      compatibilityVersion: 'compatibility-v1',
      modelVersion: 'rank-v2',
      scope: 'fixed assays',
      status: 'scored',
    },
    {
      candidateId: 'other',
      protein: 'FGHI',
      rank: null,
      score: null,
      esm2Score: null,
      modelVersion: '',
      scope: 'fixed assays',
      status: 'not_scored',
    },
  ];
  const [target, selected, other] = parse(buildScientistCsv(input));
  for (const row of [target, selected]) {
    assert.equal(row.c295_hat, '0');
    assert.equal(row.c388_hat, '-0.1234567890123456');
```





```
    assert.equal(row.c871_hat, '1.234567890123456');
    assert.equal(row.s_hat_raw, '-0.01234567890123456');
    assert.equal(row.esm2_score, '-2.987654321098765');
    assert.equal(row.esm_reference_sha256, 'd'.repeat(64));
    assert.equal(row.esm_source, 'imported');
    assert.equal(row.esm_checkpoint_sha256, 'e'.repeat(64));
    assert.equal(row.esm_compatibility_version, 'compatibility-v1');
    assert.equal(row.reference_sha256, 'a'.repeat(64));
  }
  for (const column of [
    'c295_hat',
    'c388_hat',
    'c871_hat',
    'esm_reference_sha256',
    'esm_source',
    'esm_checkpoint_sha256',
    'esm_compatibility_version',
  ]) {
    assert.equal(other[column], '');
  }
});

test('all external sites retain independent metadata and true zero, without guessing binding end',
 () => {
  const input = fixture();
  input.externalSites = [
    {
      transcript: 'external',
      position: 81,
      context: null,
      score: 0,
      bindingStart: 70,
      mismatches: 0,
      status: 'scored',
    },
    { transcript: 'external', position: 82, context, score: null, status: 'unavailable' },
  ];
  input.externalMetadata = {
    modelVersion: 'external-v1',
    referenceHash: input.reference.sha256,
    scope: 'uploaded reference',
  };
  const rows = parse(buildScientistCsv(input)).slice(1);
  assert.equal(rows.length, 2);
  assert.deepEqual(
    rows.map((row) => row.record_type),
    ['external_site', 'external_site'],
  );
  assert.deepEqual(
    rows.map((row) => row.lamar_score),
    ['0', ''],
  );
  assert.deepEqual(
    rows.map((row) => row.lamar_score_available),
    ['true', 'false'],
  );
  assert.equal(rows[0].binding_start_1based, '70');
  assert.equal(rows[0].binding_end_1based, '');
  assert.equal(rows[0].lamar_model_version, 'external-v1');
  assert.equal(rows[0].lamar_scope, 'uploaded reference');
  assert.equal(rows[0].lamar_reference_sha256, input.reference.sha256);
```





```
});

test('known reference mismatch suppresses stale LAMAR scores while retaining provenance', () => {
  const input = fixture();
  input.predictions = {
    modelVersion: 'LAMAR-v1',
    referenceSet: 'old',
    scoreType: 'ranking_score',
    predictions: [{ context, score: 0.7 }],
  };
  input.predictionMetadata = { referenceHash: 'b'.repeat(64) };
  input.externalSites = [
    { transcript: 'external', position: 9, context, score: 0.2, status: 'scored' },
  ];
  input.externalMetadata = { referenceHash: 'c'.repeat(64), modelVersion: 'old-model' };
  const rows = parse(buildScientistCsv(input));
  for (const row of rows) {
    assert.equal(row.lamar_score, '');
    assert.equal(row.lamar_score_available, 'false');
    assert.equal(row.lamar_status, 'reference_mismatch');
  }
  assert.equal(rows[0].lamar_reference_sha256, 'c'.repeat(64));
  assert.equal(rows[1].lamar_model_version, 'old-model');
  delete input.externalSites;
  const legacy = parse(buildScientistCsv(input))[0];
  assert.equal(legacy.lamar_reference_sha256, 'b'.repeat(64));
  assert.equal(legacy.lamar_score, '');
  assert.equal(legacy.lamar_status, 'reference_mismatch');
});

test('raw LAMAR scores remain negative or greater than one without normalized labels', () => {
  const input = fixture();
  input.predictions = {
    modelVersion: 'raw-lamar-v1',
    referenceSet: 'raw-population',
    scoreType: 'raw_ranking_score',
    predictions: [{ context, score: -1.234567890123456 }],
  };
  input.externalSites = [
    {
      transcript: input.target.id,
      position: input.target.position,
      context,
      score: -1.234567890123456,
      status: 'scored',
    },
    {
      transcript: 'external',
      position: 81,
      context,
      score: 2.345678901234567,
      status: 'scored',
    },
  ];
  input.externalMetadata = {
    modelVersion: 'raw-lamar-v1',
    referenceHash: input.reference.sha256,
    scope: 'uploaded reference',
    scoreType: 'raw_ranking_score',
  };
  const [target, , external] = parse(buildScientistCsv(input));
```





```
  assert.equal(target.lamar_score, '-1.234567890123456');
  assert.equal(external.lamar_score, '2.345678901234567');
  assert.equal(target.lamar_score_type, 'raw_ranking_score');
  assert.equal(external.lamar_score_type, 'raw_ranking_score');
  assert.equal(target.lamar_score_available, 'true');
  assert.equal(external.lamar_score_available, 'true');
});

test('external result target scores require the exact reported locus, even when contexts match', (
) => {
  const input = fixture();
  input.predictions = {
    modelVersion: 'raw-lamar-v1',
    referenceSet: 'raw-population',
    scoreType: 'raw_ranking_score',
    predictions: [{ context, score: 0.75 }],
  };
  input.externalMetadata = {
    modelVersion: 'raw-lamar-v1',
    referenceHash: input.reference.sha256,
    scope: 'uploaded reference',
    scoreType: 'raw_ranking_score',
  };
  input.externalSites = [
    {
      transcript: 'other-transcript',
      position: input.target.position,
      context,
      score: 0.75,
      status: 'scored',
    },
  ];
  let target = parse(buildScientistCsv(input))[0];
  assert.equal(target.lamar_score, '');
  assert.equal(target.lamar_score_available, 'false');
  assert.equal(target.lamar_status, 'site_unreported');
  input.externalSites.push({
    transcript: input.target.id,
    position: input.target.position,
    context,
    score: null,
    status: 'unscored',
  });
  target = parse(buildScientistCsv(input))[0];
  assert.equal(target.lamar_score, '');
  assert.equal(target.lamar_score_available, 'false');
  assert.equal(target.lamar_status, 'unscored');
  input.externalSites[1].score = 0;
  input.externalSites[1].status = 'scored';
  target = parse(buildScientistCsv(input))[0];
  assert.equal(target.lamar_score, '0');
  assert.equal(target.lamar_score_available, 'true');
  assert.equal(target.lamar_status, 'scored');
});

test('empty external results leave the target missing while absent external results retain legacy
context lookup', () => {
  const input = fixture();
  input.predictions = {
    modelVersion: 'legacy-lamar',
    referenceSet: 'legacy',
```





```
    scoreType: 'ranking_score',
    predictions: [{ context, score: 0.5 }],
  };
  assert.equal(parse(buildScientistCsv(input))[0].lamar_score, '0.5');
  input.externalSites = [];
  const target = parse(buildScientistCsv(input))[0];
  assert.equal(target.lamar_score, '');
  assert.equal(target.lamar_status, 'site_unreported');
  assert.equal(target.lamar_score_available, 'false');
});

test('RFC 4180 escaping preserves quotes, commas, Chinese text and embedded CRLF', () => {
  const input = fixture();
  input.target.id = '样本,"quoted"\r\nnext line';
  input.reference.name = 'RNA,reference';
  const rows = parse(buildScientistCsv(input));
  assert.equal(rows[0].target_id, input.target.id);
  assert.equal(rows[0].transcript_id, input.target.id);
  assert.equal(rows[0].reference_name, input.reference.name);
});

test('spreadsheet formula injection in string fields is neutralized, including hidden prefixes', (
) => {
  for (const payload of [
    '=1+1',
    '+SUM(A1)',
    '-1+2',
    '@SUM(A1)',
    '  =1+1',
    '\t@SUM(A1)',
    '\r\n=1+1',
    '\u0000=1+1',
  ]) {
    const input = fixture();
    input.target.id = payload;
    input.reference.name = payload;
    input.libraryVersion = payload;
    const row = parse(buildScientistCsv(input))[0];
    assert.equal(row.target_id, `'${payload}`);
    assert.equal(row.reference_name, `'${payload}`);
    assert.equal(row.library_version, `'${payload}`);
  }
});

test('nonfinite model outputs remain missing and the exporter does not infer target validity', ()
=> {
  const input = fixture();
  input.target = { id: 'short-input', sequence: 'ACGU', position: 2 };
  input.plan.sequence = null;
  input.plan.scaffold = null;
  input.externalSites = [
    {
      transcript: 'external',
      position: 1,
      context: null,
      score: NaN,
      status: 'invalid-output',
    },
  ];
```





```
  const rows = parse(buildScientistCsv(input));
  assert.equal(rows[0].site_position_1based, '2');
  assert.equal(rows[0].context_101nt, '');
  assert.equal(rows[0].protein_sequence, '');
  assert.equal(rows[0].protein_selected, 'false');
  assert.equal(rows[1].lamar_score, '');
  assert.equal(rows[1].lamar_score_available, 'false');
});
```





tests/offtarget-workflow.test.mjs

```
import { test } from 'node:test';
import assert from 'node:assert/strict';
import {
  OFFTARGET_MODEL_ID,
  emptyOffTargetState,
  targetKey,
  validateJobReceipt,
  validateOffTargetResult,
  resultMatchesInputs,
  predictionsFromResult,
} from '../lib/offtarget-workflow.ts';

const context = 'A'.repeat(50) + 'C' + 'G'.repeat(50);
const otherContext = 'U'.repeat(50) + 'C' + 'A'.repeat(50);
function fixture() {
  const target = {
    id: 'target',
    sequence: context,
    position: 51,
    motif: 'AAAAAAAA',
    bindingStart: 42,
  };
  const reference = {
    name: 'reference.fasta',
    species: 'Fixture species',
    kind: 'transcript',
    sha256: 'a'.repeat(64),
    sizeBytes: 120,
  };
  const request = {
    schemaVersion: 1,
    requestId: 'request-1',
    modelId: OFFTARGET_MODEL_ID,
    target,
    reference,
    scoreType: 'raw_ranking_score',
  };
  const result = {
    schemaVersion: 1,
    jobId: 'job-1',
    requestId: request.requestId,
    modelId: OFFTARGET_MODEL_ID,
    modelVersion: 'lamar-v1',
    referenceSha256: reference.sha256,
    species: reference.species,
    referenceKind: 'transcript',
    scope: 'uploaded-transcript-fasta',
    scoreType: 'raw_ranking_score',
    target: { ...target },
    sites: [
      { transcript: 'transcript-1', position: 51, context, score: 0, status: 'scored' },
      {
        transcript: 'transcript-2',
        position: 64,
        context: otherContext,
        score: null,
```





```
        status: 'unscored',
      },
      {
        transcript: 'short',
        position: 5,
        context: null,
        score: null,
        status: 'insufficient_context',
      },
    ],
  };
  const job = {
    id: result.jobId,
    requestId: request.requestId,
    referenceSha256: reference.sha256,
    status: 'succeeded',
  };
  const state = {
    ...emptyOffTargetState,
    file: new File(['>reference\n' + context], 'reference.fasta'),
    reference,
    species: reference.species,
    request,
    job,
    result,
  };
  return { target, reference, request, result, job, state };
}

test('zero and missing site scores retain their meanings in validated copies and prediction export
', () => {
  const { result, request } = fixture(),
    before = structuredClone(result);
  const validated = validateOffTargetResult(result, request, result.jobId);
  assert.deepEqual(
    validated.sites.map((site) => site.score),
    [0, null, null],
  );
  assert.deepEqual(predictionsFromResult(validated), {
    modelVersion: 'lamar-v1',
    referenceSet: request.reference.sha256,
    scoreType: 'raw_ranking_score',
    predictions: [{ context, score: 0 }],
  });
  validated.sites[0].transcript = 'changed-copy';
  validated.target.id = 'changed-copy';
  assert.deepEqual(result, before);
});

test('site score, context and status must agree without treating missing values as zero', () => {
  const invalid = [
    { status: 'scored', context, score: null },
    { status: 'scored', context: null, score: 0 },
    { status: 'unscored', context, score: 0 },
    { status: 'insufficient_context', context, score: null },
    { status: 'scored', context, score: NaN },
    { status: 'scored', context, score: Infinity },
    { status: 'scored', context, score: '0' },
    { status: 'scored', context: 'A'.repeat(101), score: 0 },
    { status: 'scored', context: context.toLowerCase(), score: 0 },
    { status: 'scored', context: context.slice(1), score: 0 },
```





```
    { status: 'unknown', context, score: null },
  ];
  for (const patch of invalid) {
    const { result, request } = fixture();
    result.sites = [{ transcript: 'bad', position: 51, ...patch }];
    assert.throws(
      () => validateOffTargetResult(result, request),
      undefined,
      JSON.stringify(patch),
    );
  }
  for (const field of ['context', 'score']) {
    const { result, request } = fixture();
    delete result.sites[0][field];
    assert.throws(() => validateOffTargetResult(result, request));
  }
});

test('finite raw scores outside zero to one retain their original values', () => {
  const { result, request } = fixture();
  result.sites = [
    { transcript: 'negative', position: 51, context, score: -1.25, status: 'scored' },
    {
      transcript: 'positive',
      position: 51,
      context: otherContext,
      score: 2.5,
      status: 'scored',
    },
  ];
  const validated = validateOffTargetResult(result, request);
  assert.deepEqual(
    validated.sites.map((site) => site.score),
    [-1.25, 2.5],
  );
  assert.equal(validated.scoreType, 'raw_ranking_score');
  assert.deepEqual(
    predictionsFromResult(validated).predictions.map((row) => row.score),
    [-1.25, 2.5],
  );
});

test('result provenance is bound to the exact RNA, reference, species, request and job', () => {
  const patches = [
    { referenceSha256: 'b'.repeat(64) },
    { requestId: 'request-2' },
    { species: 'Different species' },
    { jobId: 'different-job' },
    { referenceKind: 'genome' },
    { scope: 'whole-transcriptome' },
    { modelId: 'different-model' },
    { scoreType: 'probability' },
    { schemaVersion: 2 },
    { modelVersion: '' },
  ];
  for (const patch of patches) {
    const { result, request } = fixture();
    assert.throws(
      () => validateOffTargetResult({ ...result, ...patch }, request, result.jobId),
      /match|contract/,
    );
```





```
  }
  for (const patch of [
    { id: 'different' },
    { sequence: otherContext },
    { position: 52 },
    { motif: 'AAAAAAAG' },
    { bindingStart: 43 },
  ]) {
    const { result, request } = fixture();
    assert.throws(
      () =>
        validateOffTargetResult(
          { ...result, target: { ...result.target, ...patch } },
          request,
        ),
      /different RNA/,
    );
  }
});

test('site coordinates, mismatch counts and duplicate mapping keys are checked', () => {
  for (const patch of [
    { position: 0 },
    { position: 1.5 },
    { position: Number.MAX_SAFE_INTEGER + 1 },
    { bindingStart: 0 },
    { bindingStart: 2.5 },
    { mismatches: -1 },
    { mismatches: 9 },
    { transcript: '' },
  ]) {
    const { result, request } = fixture();
    result.sites = [{ ...result.sites[0], ...patch }];
    assert.throws(() => validateOffTargetResult(result, request));
  }
  const { result, request } = fixture();
  result.sites.push({ ...result.sites[0] });
  assert.throws(() => validateOffTargetResult(result, request), /Duplicate site/);
  result.sites = [result.sites[0], { ...result.sites[0], bindingStart: 2 }];
  assert.equal(validateOffTargetResult(result, request).sites.length, 2);
});

test('identical contexts share scores but retain every locus; conflicting values are rejected', ()
 => {
  const { result, request } = fixture();
  result.sites = [result.sites[0], { ...result.sites[0], transcript: 'other-locus' }];
  const valid = validateOffTargetResult(result, request);
  assert.equal(valid.sites.length, 2);
  assert.equal(predictionsFromResult(valid).predictions.length, 1);
  result.sites[1].score = 0.5;
  assert.throws(() => validateOffTargetResult(result, request), /conflicting scores/);
});

test('job receipts must match request, reference and expected opaque job id', () => {
  const { request, job } = fixture();
  assert.deepEqual(
    validateJobReceipt({ ...job, extra: 'discard' }, request, job.id),
    job,
  );
  for (const patch of [
    { id: '../other' },
```





```
    { id: 'other' },
    { requestId: 'other' },
    { referenceSha256: 'b'.repeat(64) },
    { status: 'complete' },
    { message: '' },
  ]) {
    assert.throws(() => validateJobReceipt({ ...job, ...patch }, request, job.id));
  }
});

test('changed current inputs invalidate previously accepted results', () => {
  const { state, target } = fixture();
  assert.equal(resultMatchesInputs(state, target), true);
  assert.equal(
    resultMatchesInputs({ ...state, species: `  ${state.species} ` }, target),
    true,
  );
  for (const field of ['file', 'request', 'reference', 'result']) {
    assert.equal(resultMatchesInputs({ ...state, [field]: null }, target), false, field);
  }
  for (const patch of [
    { id: 'other' },
    { sequence: otherContext },
    { position: 52 },
    { motif: 'AAAAAAAG' },
    { bindingStart: 43 },
  ]) {
    assert.notEqual(targetKey({ ...target, ...patch }), targetKey(target));
    assert.equal(resultMatchesInputs(state, { ...target, ...patch }), false);
  }
  assert.equal(
    resultMatchesInputs({ ...state, species: 'Different species' }, target),
    false,
  );
  assert.equal(
    resultMatchesInputs(
      { ...state, reference: { ...state.reference, sha256: 'b'.repeat(64) } },
      target,
    ),
    false,
  );
});

test('a replacement request cannot reactivate an older result for the same RNA and reference', ()
=> {
  const { state, target } = fixture();
  const changed = { ...state, request: { ...state.request, requestId: 'new-request' } };
  assert.equal(resultMatchesInputs(changed, target), false);
});

test('reference metadata edits cannot reactivate results from another species', () => {
  const { state, target } = fixture();
  const species = 'Different species';
  const changed = { ...state, species, reference: { ...state.reference, species } };
  assert.equal(resultMatchesInputs(changed, target), false);
});

test('matching request inputs cannot hide a stale result target or job', () => {
  const { state, target } = fixture();
  assert.equal(
    resultMatchesInputs(
```





```
      { ...state, result: { ...state.result, target: { ...target, position: 52 } } },
      target,
    ),
    false,
  );
  assert.equal(
    resultMatchesInputs(
      { ...state, result: { ...state.result, jobId: 'old-job' } },
      target,
    ),
    false,
  );
});

test('restored site evidence stays inactive until its reference is reselected and verified', () =>
 {
  const { state, target } = fixture();
  assert.equal(resultMatchesInputs({ ...state, file: null }, target), false);
  assert.equal(resultMatchesInputs(state, target), true);
});
```





tests/workflow-service.test.mjs

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { handleWorkflowRequest } from '../lib/workflow-service.ts';

const ORIGIN = 'https://studio.example';
const ENV = {
  WORKFLOW_MODEL_URL: 'https://models.example/service/',
  WORKFLOW_MODEL_TOKEN: 'server-only-secret',
};
const HEALTH = { available: true, models: { esm2: true, lamar: false } };

function request(path, init = {}) {
  return new Request(`${ORIGIN}/api/workflow${path}`, init);
}

function post(path, body = '{}', headers = {}) {
  return request(path, {
    method: 'POST',
    body,
    headers: { Origin: ORIGIN, 'Content-Type': 'application/json', ...headers },
  });
}

function streamRequest(path, chunks, contentType) {
  return request(path, {
    method: 'POST',
    headers: { Origin: ORIGIN, 'Content-Type': contentType },
    body: new ReadableStream({
      start(controller) {
        for (const chunk of chunks) controller.enqueue(new TextEncoder().encode(chunk));
        controller.close();
      },
    }),
    duplex: 'half',
  });
}

test('unconfigured health is explicitly unavailable; unrelated routes stay with Vinext', async ()
=> {
  const response = await handleWorkflowRequest(request('/health'), {});
  assert.equal(response.status, 200);
  assert.deepEqual(await response.json(), {
    available: false,
    reason: 'Model service is not configured.',
    models: { esm2: false, lamar: false },
  });
  assert.equal(response.headers.get('cache-control'), 'no-store');
  assert.equal(
    await handleWorkflowRequest(new Request(`${ORIGIN}/_vinext/image`), {}),
    null,
  );
  assert.equal(
    await handleWorkflowRequest(new Request(`${ORIGIN}/api/workflowish`), {}),
    null,
  );
  assert.equal((await handleWorkflowRequest(post('/rank'), {})).status, 503);
```





```
});

test('health validates booleans, preserves model readiness, and removes unknown fields', async ()
=> {
  const response = await handleWorkflowRequest(request('/health'), ENV, {
    fetch: async (url, init) => {
      assert.equal(url.href, 'https://models.example/service/health');
      assert.equal(init.method, 'GET');
      assert.equal(init.redirect, 'error');
      return Response.json({ ...HEALTH, secret: 'must not appear' });
    },
  });
  assert.deepEqual(await response.json(), HEALTH);
  for (const invalid of [
    { available: 'true', models: HEALTH.models },
    { available: true, models: { esm2: true } },
    { ...HEALTH, reason: 42 },
  ]) {
    const failed = await handleWorkflowRequest(request('/health'), ENV, {
      fetch: async () => Response.json(invalid),
    });
    assert.equal(failed.status, 502);
    assert.equal((await failed.json()).available, false);
  }
});

test('upstream configuration accepts only HTTPS without credentials, query, or fragment', async ()
 => {
  for (const url of [
    'http://models.example',
    'https://user:password@models.example',
    'https://models.example?token=secret',
    'https://models.example#fragment',
    'not a URL',
  ]) {
    let called = false;
    const response = await handleWorkflowRequest(
      post('/rank'),
      { WORKFLOW_MODEL_URL: url },
      {
        fetch: async () => {
          called = true;
          return Response.json({});
        },
      },
    );
    assert.equal(response.status, 503);
    assert.equal(called, false);
    assert.equal((await response.text()).includes(url), false);
  }
});

test('mutations reject cross-origin and missing Origin before any upstream call', async () => {
  let calls = 0;
  const options = {
    fetch: async () => {
      calls++;
      return Response.json({});
    },
  };
  for (const origin of [
```





```
    'https://attacker.example',
    'null',
    `${ORIGIN}.attacker.example`,
  ]) {
    assert.equal(
      (await handleWorkflowRequest(post('/rank', '{}', { Origin: origin }), ENV, options))
        .status,
      403,
    );
  }
  assert.equal(
    (
      await handleWorkflowRequest(
        request('/rank', {
          method: 'POST',
          body: '{}',
          headers: { 'Content-Type': 'application/json' },
        }),
        ENV,
        options,
      )
    ).status,
    403,
  );
  assert.equal(calls, 0);
});

test('rank forwards only server-selected credentials and strips upstream response headers', async
() => {
  const response = await handleWorkflowRequest(
    post('/rank', '{"candidateIds":["puf-1"]}', {
      Authorization: 'Bearer browser-secret',
      Cookie: 'session=browser-cookie',
      'X-Upstream-URL': 'https://attacker.example',
    }),
    ENV,
    {
      fetch: async (url, init) => {
        assert.equal(url.href, 'https://models.example/service/rank');
        assert.equal(init.headers.get('authorization'), 'Bearer server-only-secret');
        assert.equal(init.headers.get('cookie'), null);
        assert.equal(init.headers.get('origin'), null);
        assert.equal(init.headers.get('x-upstream-url'), null);
        assert.deepEqual(JSON.parse(init.body), { candidateIds: ['puf-1'] });
        return Response.json(
          { ranks: [] },
          {
            status: 201,
            headers: { 'Set-Cookie': 'upstream=secret', 'X-Internal-URL': 'private' },
          },
        );
      },
    },
  );
  assert.equal(response.status, 201);
  assert.deepEqual(await response.json(), { ranks: [] });
  assert.equal(response.headers.get('set-cookie'), null);
  assert.equal(response.headers.get('x-internal-url'), null);
});

test('an unset server token never forwards the caller Authorization', async () => {
```





```
  await handleWorkflowRequest(
    post('/rank', '{}', { Authorization: 'Bearer browser-secret' }),
    { WORKFLOW_MODEL_URL: ENV.WORKFLOW_MODEL_URL },
    {
      fetch: async (_, init) => {
        assert.equal(init.headers.get('authorization'), null);
        return Response.json({});
      },
    },
  );
});

test('invalid paths, encoded traversal, queries, IDs, and methods cannot redirect upstream', async
 () => {
  let calls = 0;
  const options = {
    fetch: async () => {
      calls++;
      return Response.json({});
    },
  };
  for (const path of [
    '/jobs/a%2Fb',
    '/jobs/%252e%252e',
    '/jobs/%2e%2e/rank',
    '/jobs/a.b',
    '/jobs/_bad-id',
    '/jobs/-bad-id',
    '/jobs/a/result/extra',
    '/other',
    `/jobs/${'a'.repeat(129)}`,
  ]) {
    const response = await handleWorkflowRequest(request(path), ENV, options);
    assert.ok([404, 405].includes(response.status), path);
  }
  assert.equal(
    (
      await handleWorkflowRequest(
        request('/jobs/valid-id?url=https://attacker.example'),
        ENV,
        options,
      )
    ).status,
    400,
  );
  const method = await handleWorkflowRequest(request('/rank'), ENV, options);
  assert.equal(method.status, 405);
  assert.equal(method.headers.get('allow'), 'POST');
  assert.equal(calls, 0);
});

test('job polling and result paths map only to the configured service', async () => {
  for (const suffix of ['', '/result']) {
    const response = await handleWorkflowRequest(
      request(`/jobs/job_123-AbC${suffix}`),
      ENV,
      {
        fetch: async (url, init) => {
          assert.equal(
            url.href,
            `https://models.example/service/jobs/job_123-AbC${suffix}`,
```





```
          );
          assert.equal(init.method, 'GET');
          return Response.json({ id: 'job_123-AbC' });
        },
      },
    );
    assert.equal(response.status, 200);
  }
});

test('known request lengths over each cap are rejected before forwarding', async () => {
  let calls = 0;
  const options = {
    fetch: async () => {
      calls++;
      return Response.json({});
    },
  };
  assert.equal(
    (
      await handleWorkflowRequest(
        post('/rank', '{}', { 'Content-Length': String(2 * 1024 * 1024 + 1) }),
        ENV,
        options,
      )
    ).status,
    413,
  );
  assert.equal(
    (
      await handleWorkflowRequest(
        post('/jobs', 'x', {
          'Content-Type': 'multipart/form-data; boundary=test',
          'Content-Length': String(100 * 1024 * 1024 + 64 * 1024 + 1),
        }),
        ENV,
        options,
      )
    ).status,
    413,
  );
  assert.equal(
    (
      await handleWorkflowRequest(
        post('/rank', '{}', { 'Content-Length': 'abc' }),
        ENV,
        options,
      )
    ).status,
    400,
  );
  assert.equal(calls, 0);
});

test('multipart envelope allows 100 MiB plus 64 KiB while retaining bounded memory', async () => {
  const chunk = new Uint8Array(64 * 1024);
  const limit = 100 * 1024 * 1024 + 64 * 1024;
  let remaining = limit;
  const incoming = request('/jobs', {
    method: 'POST',
    duplex: 'half',
```





```
    headers: {
      Origin: ORIGIN,
      'Content-Type': 'multipart/form-data; boundary=test',
      'Content-Length': String(limit),
    },
    body: new ReadableStream({
      pull(controller) {
        if (!remaining) {
          controller.close();
          return;
        }
        controller.enqueue(chunk);
        remaining -= chunk.byteLength;
      },
    }),
  });
  const response = await handleWorkflowRequest(incoming, ENV, {
    fetch: async (_, init) => {
      const reader = init.body.getReader();
      let received = 0;
      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        received += value.byteLength;
      }
      assert.equal(received, limit);
      return Response.json({ id: 'envelope-accepted' }, { status: 202 });
    },
  });
  assert.equal(response.status, 202);
});

test('unknown JSON length is byte-capped before any upstream call', async () => {
  let calls = 0;
  const response = await handleWorkflowRequest(
    streamRequest('/rank', ['{"x":"', 'éééé', '"}'], 'application/json'),
    ENV,
    {
      jsonLimitBytes: 12,
      fetch: async () => {
        calls++;
        return Response.json({});
      },
    },
  );
  assert.equal(response.status, 413);
  assert.equal(calls, 0);
});

test('multipart upload streams intact without forwarding browser headers', async () => {
  const form = new FormData();
  form.append('metadata', JSON.stringify({ name: 'fixture' }));
  form.append(
    'fasta',
    new File(['>fixture\nACGT\n'], 'fixture.fasta', { type: 'text/plain' }),
  );
  const incoming = request('/jobs', {
    method: 'POST',
    body: form,
    headers: { Origin: ORIGIN, Cookie: 'private' },
  });
```





```
  const response = await handleWorkflowRequest(incoming, ENV, {
    fetch: async (url, init) => {
      assert.equal(url.href, 'https://models.example/service/jobs');
      assert.ok(init.body instanceof ReadableStream);
      assert.equal(init.headers.get('cookie'), null);
      const received = await new Response(init.body, {
        headers: init.headers,
      }).formData();
      assert.equal(received.get('metadata'), '{"name":"fixture"}');
      assert.equal(await received.get('fasta').text(), '>fixture\nACGT\n');
      return Response.json({ id: 'fixture-job' }, { status: 202 });
    },
  });
  assert.equal(response.status, 202);
});

test('unknown multipart length overflow aborts upstream and returns 413', async () => {
  let signal;
  const response = await handleWorkflowRequest(
    streamRequest(
      '/jobs',
      ['123456', '789012', '345678'],
      'multipart/form-data; boundary=test',
    ),
    ENV,
    {
      multipartLimitBytes: 10,
      fetch: async (_, init) => {
        signal = init.signal;
        await new Response(init.body).arrayBuffer();
        return Response.json({ id: 'must not succeed' });
      },
    },
  );
  assert.equal(response.status, 413);
  assert.equal(signal.aborted, true);
});

test('an upstream acknowledgement cannot bypass an unconsumed upload', async () => {
  const response = await handleWorkflowRequest(
    streamRequest('/jobs', ['1234', '5678'], 'multipart/form-data; boundary=test'),
    ENV,
    {
      multipartLimitBytes: 6,
      requestTimeoutMs: 5,
      fetch: async () => Response.json({ id: 'too-early' }, { status: 202 }),
    },
  );
  assert.equal(response.status, 504);
});

test('validated JSON retains its original numeric representation', async () => {
  const body = '{"identifier":9007199254740993,"small":1e-1000}';
  const response = await handleWorkflowRequest(post('/rank', body), ENV, {
    fetch: async (_, init) => {
      assert.equal(init.body, body);
      return Response.json({});
    },
  });
  assert.equal(response.status, 200);
});
```





```
418
419 test('content type and malformed JSON are rejected without forwarding', async () => {
420   let calls = 0;
421   const options = {
422     fetch: async () => {
423       calls++;
424       return Response.json({});
425     },
426   };
427   for (const body of ['{', '[]', 'null'])
428     assert.equal(
429       (await handleWorkflowRequest(post('/rank', body), ENV, options)).status,
430       400,
431     );
432   assert.equal(
433     (
434       await handleWorkflowRequest(
435         post('/rank', '{}', { 'Content-Type': 'text/plain' }),
436         ENV,
437         options,
438       )
439     ).status,
440     415,
441   );
442   assert.equal(
443     (
444       await handleWorkflowRequest(
445         post('/jobs', '{}', { 'Content-Type': 'multipart/form-data' }),
446         ENV,
447         options,
448       )
449     ).status,
450     415,
451   );
452   assert.equal(calls, 0);
453 });
454
455 test('upstream failures preserve error status without leaking bodies or retrying POST', async () =
  > {
456   let calls = 0;
457   const response = await handleWorkflowRequest(post('/rank'), ENV, {
458     fetch: async () => {
459       calls++;
460       return new Response('https://models.example server-only-secret', { status: 429 });
461     },
462   });
463   assert.equal(response.status, 429);
464   assert.deepEqual(await response.json(), { error: 'Model service request failed.' });
465   assert.equal(calls, 1);
466   const network = await handleWorkflowRequest(request('/health'), ENV, {
467     fetch: async () => {
468       throw new Error('https://models.example server-only-secret');
469     },
470   });
471   assert.equal(network.status, 502);
472   assert.deepEqual(await network.json(), {
473     available: false,
474     reason: 'Model service could not be reached.',
475     models: { esm2: false, lamar: false },
476   });
477 });
```





```

test('health and normal request timeouts abort a single upstream request', async () => {
  for (const isHealth of [true, false]) {
    let calls = 0;
    let signal;
    const response = await handleWorkflowRequest(
      isHealth ? request('/health') : post('/rank'),
      ENV,
      {
        healthTimeoutMs: 5,
        requestTimeoutMs: 5,
        fetch: async (_, init) => {
          calls++;
          signal = init.signal;
          return new Promise(() => {});
        },
      },
    );
    assert.equal(response.status, 504);
    assert.equal(signal.aborted, true);
    assert.equal(calls, 1);
    const message = await response.text();
    assert.equal(message.includes('server-only-secret'), false);
    assert.equal(message.includes('models.example'), false);
  }
});

test('non-JSON and malformed health responses never imply readiness', async () => {
  for (const upstream of [
    new Response('<html>failure</html>'),
    new Response('{', { headers: { 'Content-Type': 'application/json' } }),
    Response.json({ available: true }),
  ]) {
    const response = await handleWorkflowRequest(request('/health'), ENV, {
      fetch: async () => upstream,
    });
    assert.equal(response.status, 502);
    assert.equal((await response.json()).available, false);
  }
});
```





LICENSE

```
MIT License

Copyright (c) 2026 Patrick Star

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```



